

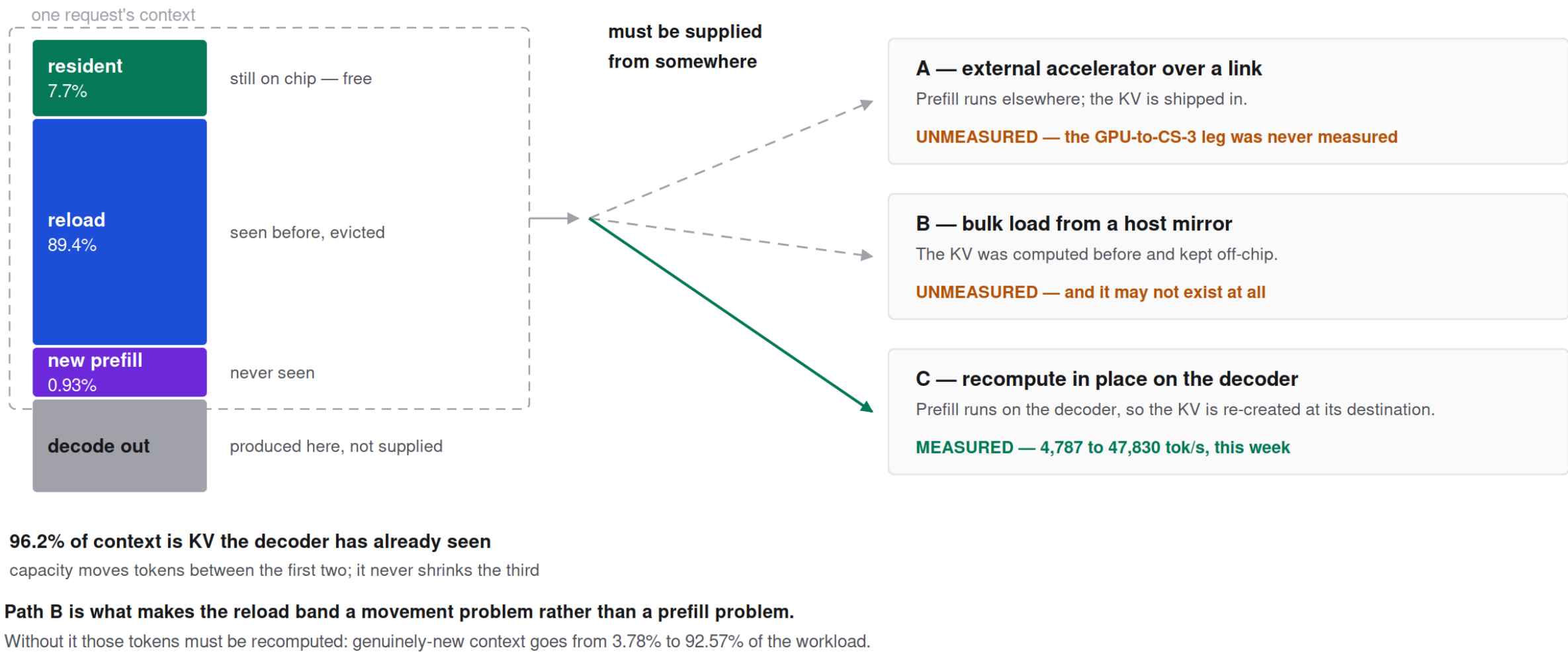
WEEKLY COLLABORATOR DISCUSSION

Where a Wafer Fits in Agentic Inference

96% of an agentic call's context is KV the decoder has already seen.
What it costs to put that back in front of the scan decides everything else.

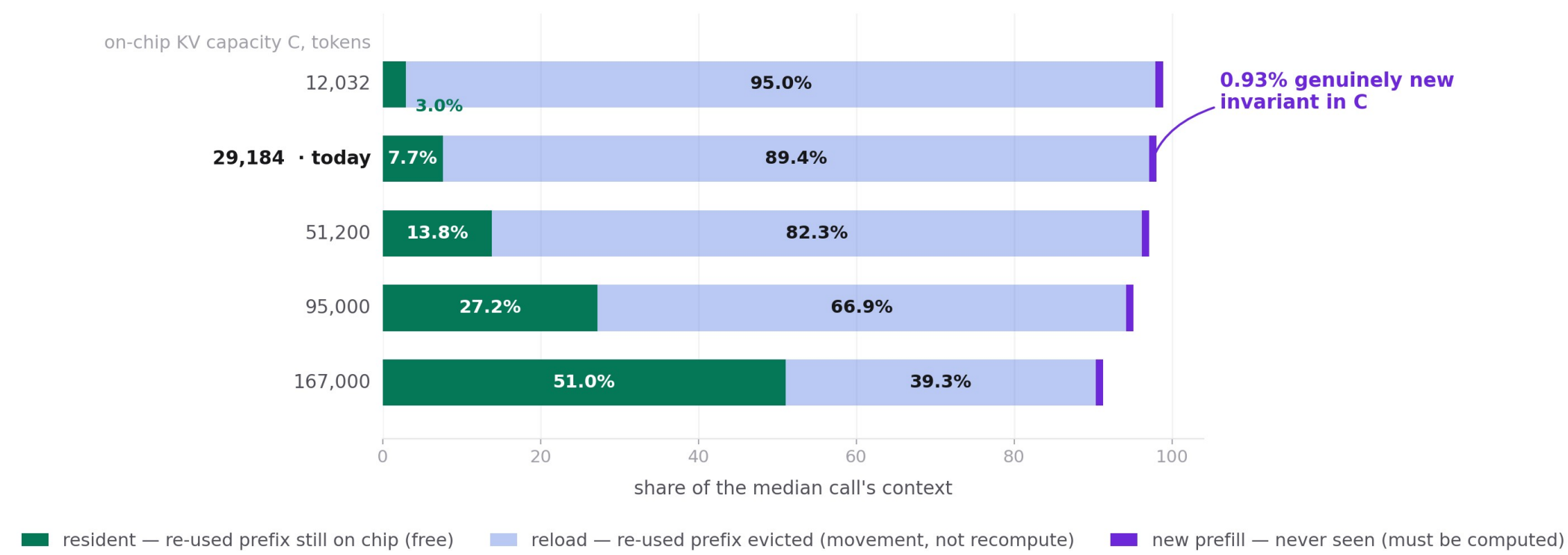
MODEL	RATES	SCOPE	DATE
Qwen3-4B	Device-measured, 0.85 GHz	Fits · doesn't · not yet	2026-09-06

Four kinds of token, and only three ways to supply them



Evicted prefix needs movement only, not recomputation. Only context the decoder has never seen must actually be prefilled — and those two cost an order of magnitude apart, so collapsing them hides the answer.

Only 0.9% of an agentic call is context the decoder has never seen



Claude Code, 46,650 calls, 512-token block LRU, anyblock scoring. Per-category medians, so rows do not sum to exactly 100%.

At today's 29,184-token ceiling the median call is 28,672 tokens resident, 165,376 re-used but evicted, and 1,820 genuinely new. Capacity trades the evicted band for the resident one and never touches the third.

Whether a host KV mirror exists changes the workload by 24×

Same trace, same replay, one assumption changed. Claude Code at today's 29,184-token capacity.

QUANTITY	WITH A HOST MIRROR	WITHOUT A MIRROR
Median per-call new prefill	1,820 tok	175,704 tok
Context tokens genuinely new	3.78%	92.57%
What the bottleneck becomes	Moving seen KV back	Recomputing seen KV
What a faster prefiller touches	3.78% of tokens	The whole workload
Is that path measured?	No — the link was never measured	Yes — 4,787 to 47,830 tok/s

Whether decoded KV re-enters context is a platform choice, not a law

TRACE	IS DECODED KV RE-USED?	CONSEQUENCE FOR ON-CHIP KV
Claude Code — agentic, one user, one month	~0% — thinking is stripped before the next call	Long decode consumes context and produces almost nothing keepable
ServeGen — reasoning, multi-tenant, Feb 2025	83.4% of turn pairs re-send the full previous output, reasoning included	Decoded KV is worth keeping; the 29K and 51K rungs matter there
Mooncake — chat and tool-agent, 2024	No thinking at all; output capped at 2,000 tokens	Informs the input side only — prefix sharing and arrival gaps

Three conditions — and the third is a property of the model, not the chip

CONDITION	EVIDENCE	CLASS
Decode is at least ~5% of newly prefilled tokens	Claude Code median turn sits at 63%	measured (trace)
The user waits per turn, with a large reasoning budget	5,838 decode tokens per turn, 75% hidden reasoning	measured (trace)
The decoder doubles as the prefiller	4.29–4.53× the native prefill kernel at 2K, 1.91–2.02× at 8K	measured (device)
But thinking length follows the model class	The shape evidence is frontier-model; a 4B's own agentic trace does not exist	Le's reading

Five ways the wafer loses

SITUATION

WHAT HAPPENS

No extended thinking in the turn

Parity — the per-turn advantage disappears entirely

Large new prefill, little decode

About 8× slower: first turn, repository dump, big tool output

Throughput per dollar

The GPU keeps a 15–60× margin under every lever set (modelled)

Multi-tenant aggregation

Decode-stream concurrency p50 = 1; filling S stages needs about 12·S in-turn users

Two wafers on chat-shaped traffic

Both Mooncake traces come out net negative at every prefill rate

Meeting the fit condition needs a bigger model — which needs more wafers

MODEL	WAFERS	RESIDENT CONTEXT	AGENTIC FIT
Qwen3-4B bf16	1	32K — 29,184 measured	Tool use; weak on SWE
Qwen3-8B bf16	1	24K	—
Qwen3.5-9B, dense / 3:1 hybrid	2	62K / 247K	Agentic-tuned
GPT-OSS-20B MXFP4	4	863K	Codex-class; reasoning-effort knob
Qwen3-235B-A22B bf16	12	39K — only 3K per wafer	Weight-minimum: the wrong deployment

"Does the whole request fit" is the wrong test

Capacity trades evicted KV for resident KV; it never reduces what must be prefilled.

ON-CHIP KV CAPACITY	RESIDENT	RELOAD	NEW PREFILL
12,032	2.96%	94.96%	0.93%
29,184 — today, measured	7.65%	89.44%	0.93%
51,200	13.83%	82.35%	0.93%
95,000	27.25%	66.92%	0.93%
167,000	51.04%	39.27%	0.93%

NOT YET → MEASURED · THIS WEEK'S THROUGHPUT WORK

Four stage granularities on silicon: 5.7×, paid for in context

One wafer, single request, forced prefill at 2K. Cycles measured, tok/s at 0.85 GHz.

Context caps are the pre-stride-lift i8 caps — see notes.

LAYOUT	STAGES	CYCLES/TOKEN	TOK/S	CONTEXT CAP
A — 4 rows × 256 ² , 9 layers per block	8	101,670	8,360	29,184
L = 4 — 9 rows × 256×128	18	48,872	17,392	16,256
L = 2 — 18 rows × 256×64	36	27,348	31,081	8,128
L = 1 — 36 rows × 256×32	72	17,771	47,830	4,064

Ten lines removed the field that was capping context

A memory descriptor's stride is a signed 8-bit field, so context was capped at 127 × the block's rows. Ten lines replace that one strided K-cache write with four scalar stores: 1,000 device records byte-identical, +0.05% cycles.

RUNG	OLD STRIDE CAP	COMPILED CEILING	LIFT
L = 1 — 32 rows, 1 layer	4,064	12,032	2.96×
L = 2 — 64 rows, 2 layers	8,128	18,176	2.24×
L = 4 — 128 rows, 4 layers	16,256	24,832	1.53×
A — 256 rows, 9 layers	32,512	26,112	none — SRAM-bound

Boot-only routing code moved to the host: 5.6 KB back on every compute PE

comm_mod.init derived the routes once at boot, then stayed resident for the whole run.
The host now computes them per PE and ships 68 B.

REGION	BASELINE	ROUTE TABLE	FREED
Decode ATTN — 4 × 65,536 PE	39,328 B	34,080 B	−5,248 B
Decode FFN — 4 × 65,536 PE	34,944 B	30,272 B	−4,672 B
ATTN role image, .text	22,892 B	17,660 B	−5,232 B
FFN role image, .text	11,316 B	6,640 B	−4,676 B
HT head / HT tail	27,360 / 45,584 B	unchanged	0
Compile-only context ceiling	23,040 tok	30,464 tok	1.32×

NOT YET · THE SAME 36 LAYERS, PLACED THREE WAYS

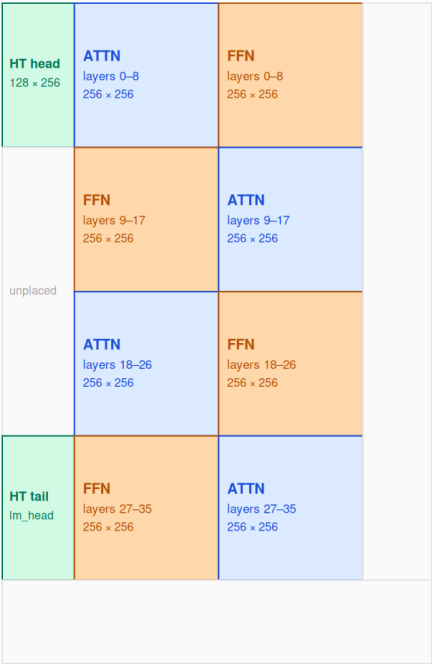
What the three layouts in the table actually look like on the wafer

Qwen3-4B on one WSE-3: three placements of the same 36 layers

Fabric 762 × 1172 PEs, drawn to scale. Blue = attention role, amber = FFN role, green = HT head/tail, grey = unplaced. Cycles measured; tok/s at 0.85 GHz.

A · today — role-split, 4 rows

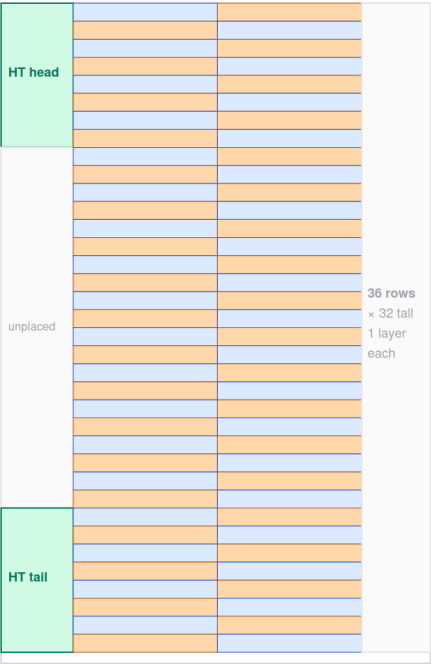
8 blocks of 256² · 9 layers per block · 8 stages · 524K PEs



8 stages · 101,670 cycles/token · 8,360 tok/s
Both roles pinned to the same 256 rows, so the FFN gets rows it does not need. Context 26,112 compiled in this tree; 29,184 was measured on an earlier one.
MEASURED, byte-identical.

C · L = 1 — one layer per block, one lane

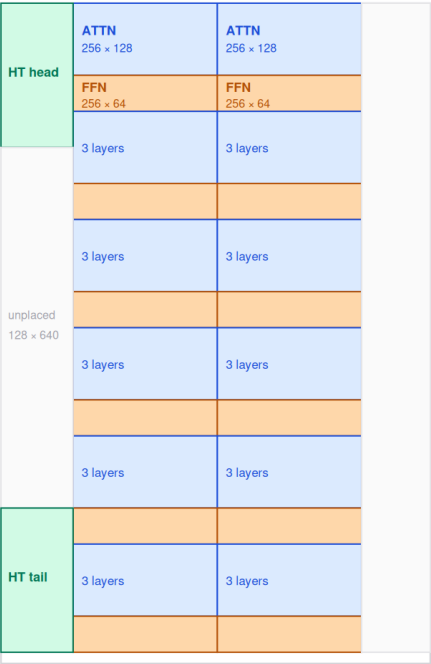
36 rows × (ATTN 256×32 + FFN 256×32) · 72 stages · 590K PEs



72 stages · 17,771 cycles/token · 47,830 tok/s
5.7× A. Only one lane of 256-wide blocks fits — a lane is 512 columns plus the HT band and strips.
Context 12,032 compiled after the stride lift, from 4,064.
MEASURED, gate passed.

Stacked · two lanes, heights free

2 lanes × 6 bands × (ATTN 256×128 over FFN 256×64) · 24 stages · 590K PEs



24 stages · ≈ 36,602 cycles/token · ≈ 23,223 tok/s
Stacking frees the two heights, so the FFN gives up rows it does not need: −1.3% on attention's own compute, +4.1% for the strip hop. Same area and ceiling as L = 4, 1.34× its rate.
PROJECTED from a measured three-layer rig.

Today's placement pins both roles to the same 256 rows. L = 1 cuts the wafer into 36 thin rows, one layer each. Stacking puts the two roles in one column so their heights can differ, paying for the halved width with a second lane.

Side by side against stacked, across the layers-per-block ladder

One wafer, one request, forced prefill at 2K, equal wafer area. Cycles measured, tok/s at 0.85 GHz. Side-by-side caps are compiled and pinned; no stacked ceiling has been compiled, so none is shown.

LAYOUT	CYCLES/TOKEN	TOK/S	CAP, 18 STRIDE	CAP, AFTER LIFT
A — 8 stages, H = 256	101,670	8,360	29,184	26,112 · earlier tree 29,184
L = 4 — 18 stages, H = 128	48,872	17,392	16,256	24,832
L = 2 — 36 stages, H = 64	27,348	31,081	8,128	18,176
L = 1 — 72 stages, H = 32	17,771	47,830	4,064	12,032
Stacked ×2, ATTN 128 FFN 64 — 24 st.	≈ 36,602	≈ 23,223	16,256	—
Stacked ×2, ATTN 96 FFN 32 — 36 st.	≈ 26,042	≈ 32,639	12,192	—
Stacked ×2, ATTN 32 FFN 32 — 72 st.	≈ 18,238	≈ 46,606	4,064	—

The two-wafer experiment, and where the 344 μs per token goes

Qwen3-4B split by layer across two CS-3s, bsz 1, byte-exact over 4,096 steps. Each token crosses the boundary twice.

LEG	PER TOKEN	NOTE
SDK direct-stream H2D + D2H, both wafers	$\approx 250\ \mu\text{s}$	$\approx 125\ \mu\text{s}$ per wafer
On-wafer host-facing gather / fan-out columns	$\approx 45\ \mu\text{s}$	set by hop count, not bytes
Host work — embedding copy 13.3, enqueue 5.3	$19\ \mu\text{s}$	per token
RDMA wire and wait	$14\ \mu\text{s}$	in-run wire residual $3.0\ \mu\text{s}$
Attributed total	328 of 344 μs	closes at 99.2%
One wafer $\rightarrow$ two wafers	$1,036.2 \rightarrow 1,380.2\ \mu\text{s}$	+33%, $n = 3$, sd 0.38%

DECIDE

What would move the argument

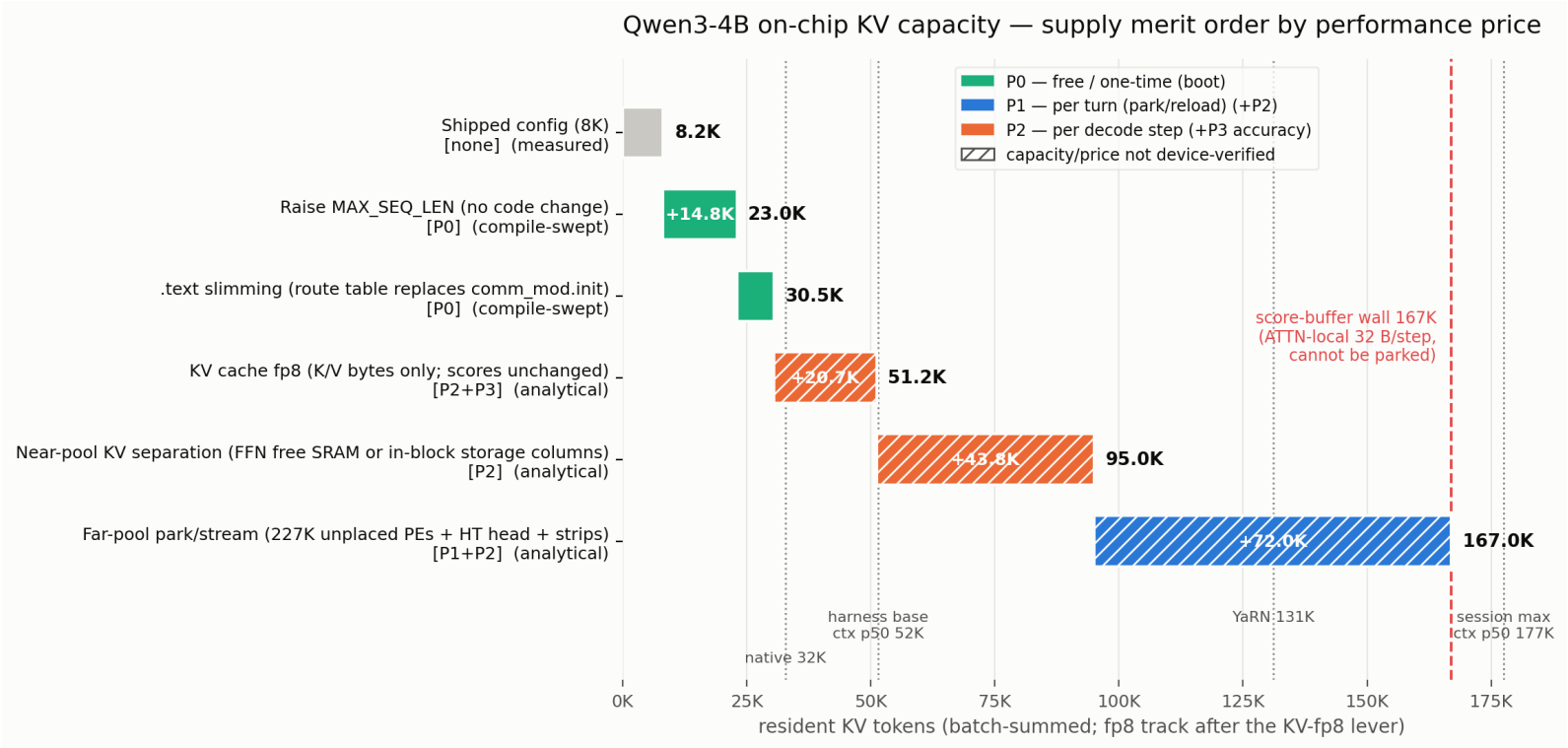
Measure first	The GPU-to-CS-3 link, and a 4B GPU curve. Both unmeasured
Design call	Is a host KV mirror part of the deployment? It moves the workload 24×
Kernel	Cut (iii): 1.71× at 24,384 context, blocked on the A1 stall
Layout	The second stacked lane: 1.34×, needs the multi-lane serpentine
Capacity	Stream KV out of idle FFN SRAM: 36,800 tokens stranded today
Workload	A same-model agentic trace on a 4B-class model

A

Appendix

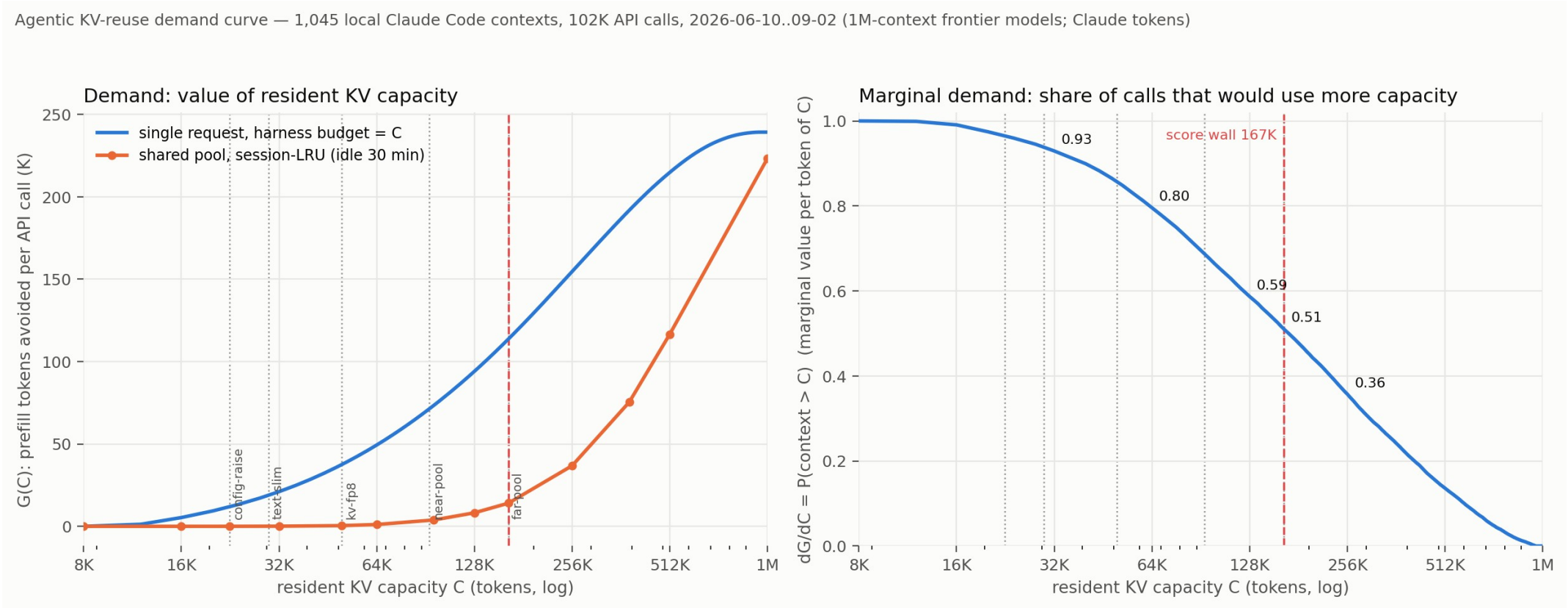
Every experimental result in the session report — one slide each, figure or table, with the question it answered.

SRAM levers are not comparable by size — they are ranked by when the price is paid



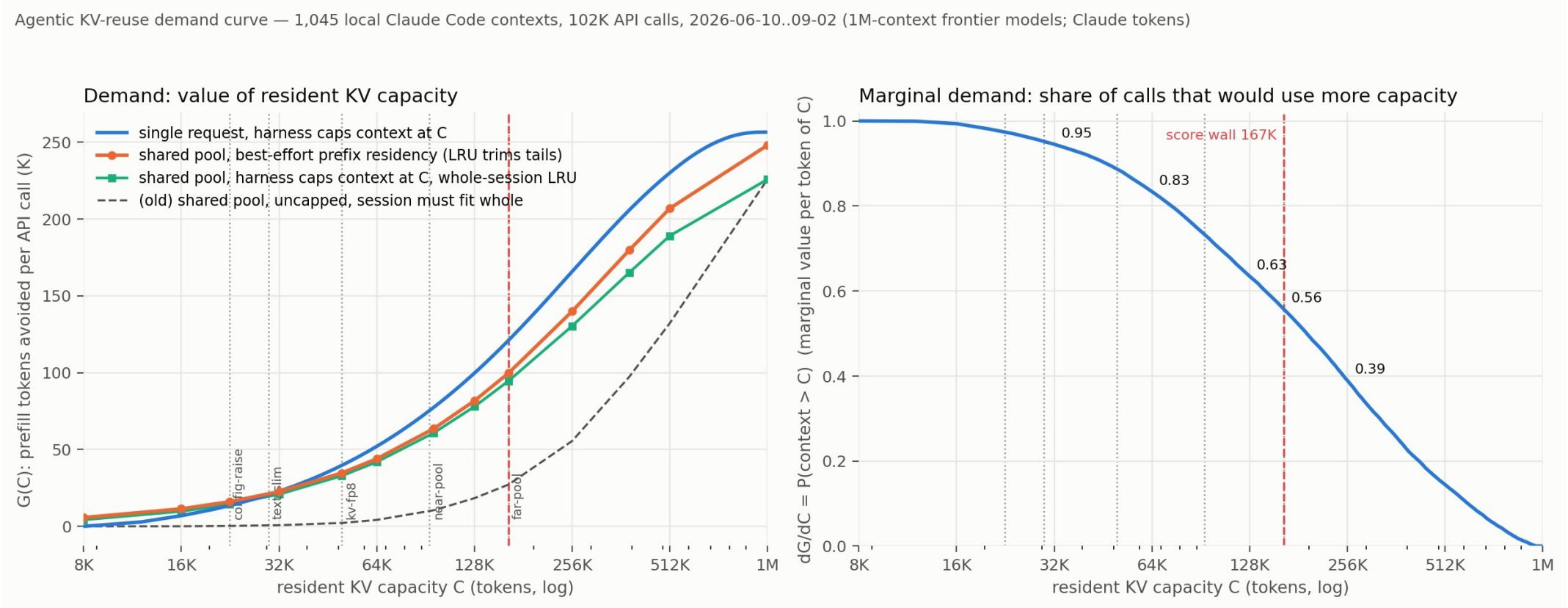
A merit order of the KV-capacity levers: x is resident KV tokens, each block is a lever in price order, colour is when the cost is paid (P0 boot, P1 per turn, P2 per decode step, P3 accuracy), hatching means not device-verified, and the red line is the 167K score-buffer wall. Produced from machine-readable price cards written for the SRAM-performance exchange methodology. It settled that there is no single formula: prices in different currencies are comparable only under a stated workload profile. Analytical, built on compiled constants.

The first demand curve was wrong: calling an over-capacity session infeasible collapsed the shared-pool line



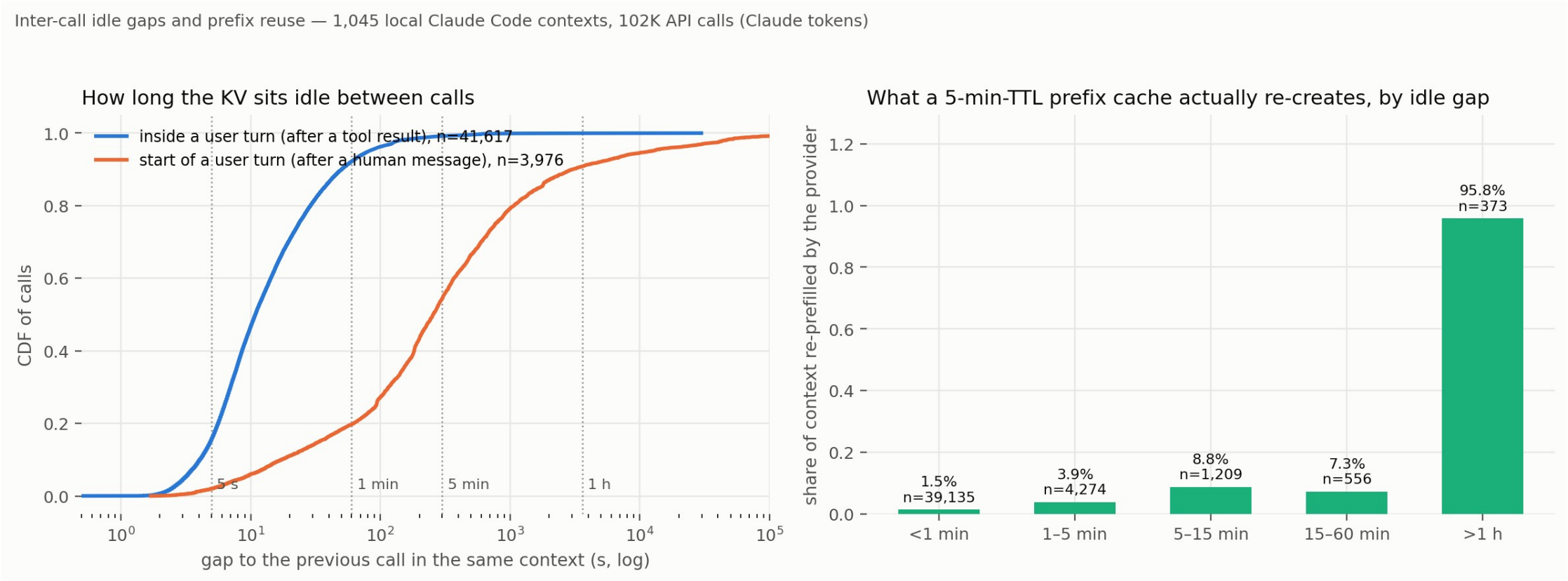
Version 1 of G(C), the prefill tokens avoided per call as a function of resident KV capacity C, replayed on 1,048 local Claude Code contexts. Its shared-pool line collapses below 256K, which is an artifact of declaring any session larger than C infeasible rather than a property of the workload. Kept for the record only; F3 is the corrected curve. Measured trace replay with an analytical residency model.

How much prefill a resident-KV capacity actually avoids, and what the next rung is worth



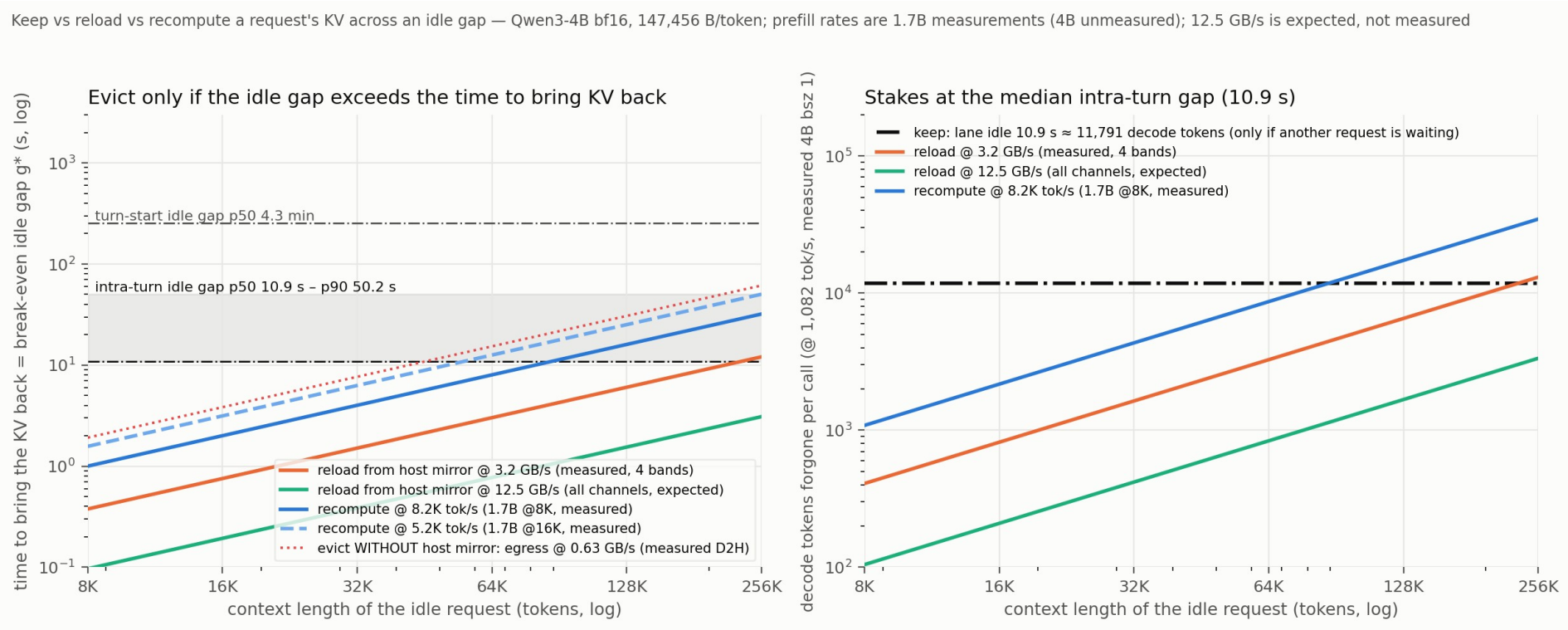
Version 2 replaces the infeasibility rule with best-effort prefix residency: an LRU trims tails and any missing prefix is recomputed, which lands the shared-pool line within 15-20% of the single-request curve. The marginal value of capacity is $dG/dC = P(\text{context} > C)$ — 0.93 at 32K, 0.80 at 64K, 0.51 at 167K. Measured trace replay of 46,650 de-duplicated API calls.

The provider keeps KV about an hour, and intra-turn gaps are 10.9 s — keeping across a turn is unconditionally right



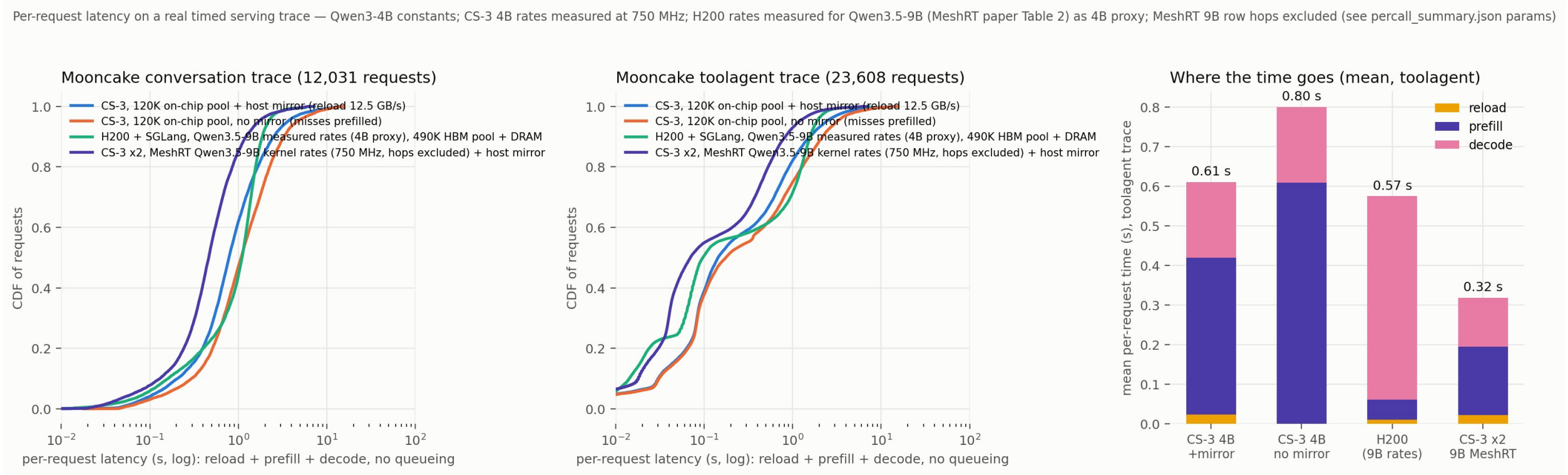
Idle-gap distributions inside a turn and at turn starts, next to the share of context the provider re-created as a function of gap length. It re-created 3.7% of context for gaps under five minutes and 95.9% beyond an hour. Measured directly from the usage fields of 46,650 de-duplicated Claude Code API calls.

Recompute is never the right recovery above ~25K tokens; without a host mirror, eviction is the most expensive option



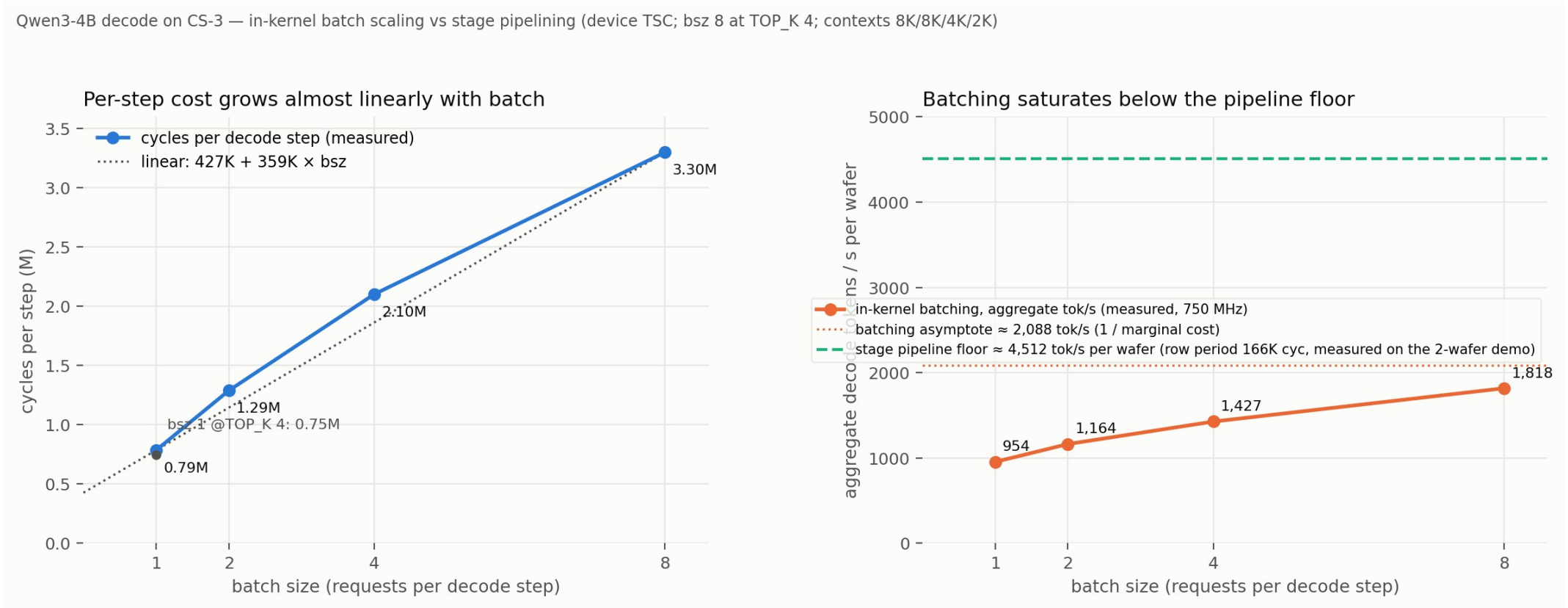
Break-even idle gap for the three recoveries — keep, reload, recompute — against context length. With a host mirror at the 12.5 GB/s design rate, reloading any context up to 256K takes less than the median intra-turn gap; without one, the measured 1.37-1.44 GB/s egress makes eviction the worst option. Decode tok/s cancels in keep-versus-reload: it sets the stakes, not the crossover. Measured gaps and measured link rates, modelled economics.

On a real serving trace the pool size is invisible — and with measured GPU rates one 4B wafer no longer wins per request



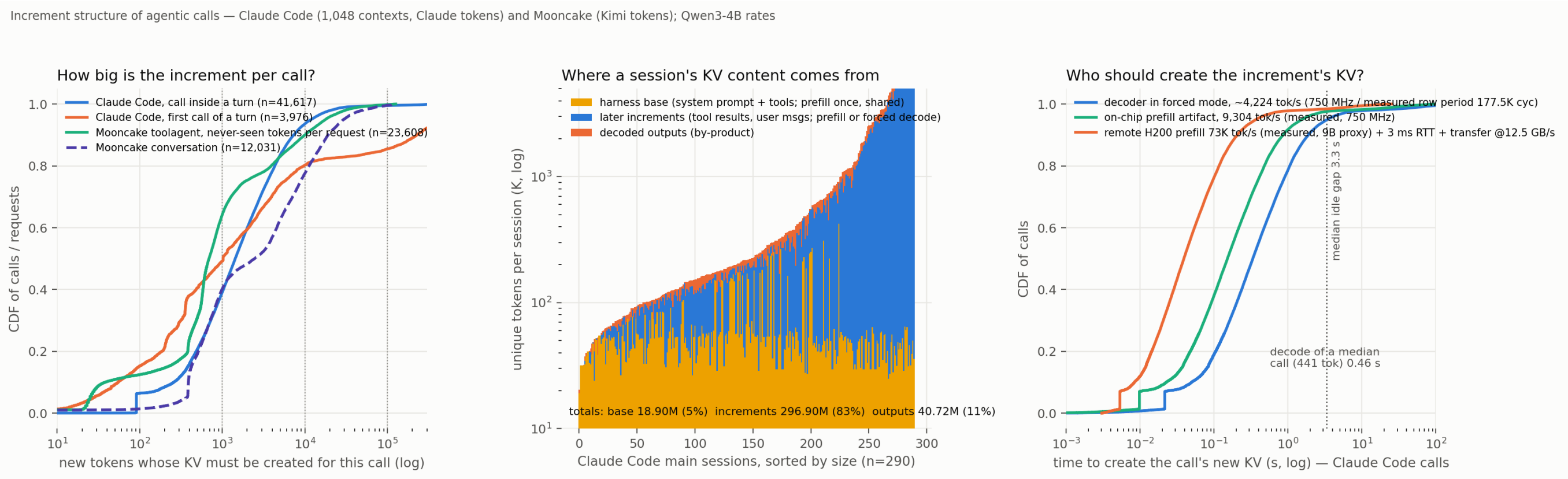
Per-request latency on the Mooncake FAST'25 conversation and toolagent traces under a 512-token block-LRU prefix cache. Cache hit rate is flat from 32K to 1M tokens and only reaches its infinite-cache value at 16-64M. The first pass had CS-3 about 2x faster per request; revised with measured H200 rates it loses on prefill-heavy requests, because the GPU's 8x faster prefill outweighs the wafer's faster decode. Measured trace, modelled single-stream latency.

Batching alone saturates the wafer at ~2,370 tok/s; pipelining requests across stages is the better multiplier



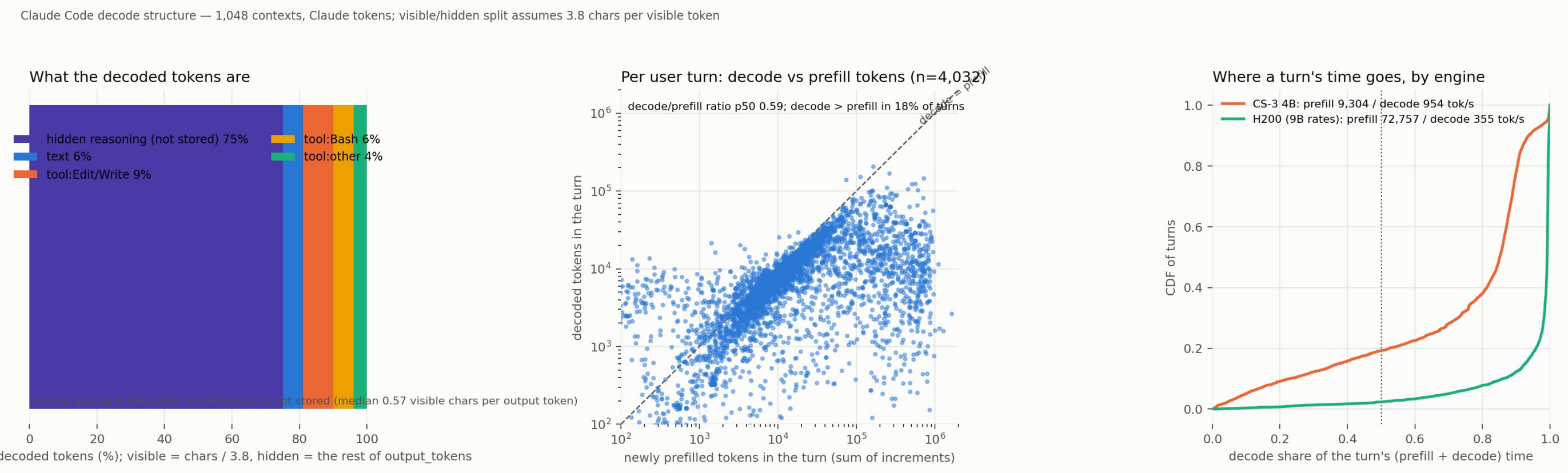
Measured decode cycles per step at batch 1, 2, 4 and 8 on CS-3, fitted as $427K + 359K \times bsz$, plotted against the two-wafer demo's pipelined per-wafer period of 166K cycles. Because the batch-independent part is only ~0.43M cycles, in-kernel batching asymptotes at $0.85e9/359K = \sim 2,370$ tok/s per wafer, while stage pipelining reaches ~5,120. Measured on-device TSC.

KV is 90-95% prefill-origin: the decoder's own output is a tenth of unique context, and tool results are the bulk



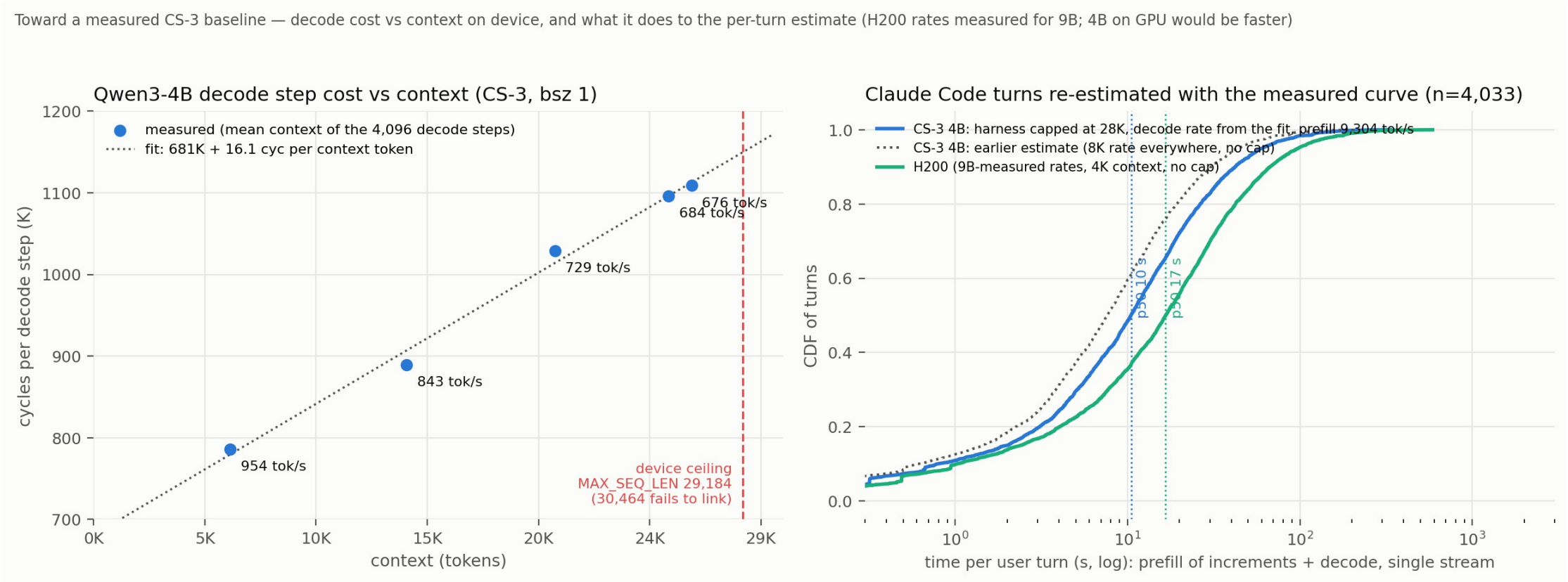
The increment structure of agentic calls — new tokens per call, what those tokens are, and where a session's unique context comes from. Decode by-products are 10% of unique context in Claude Code sessions and 4-5% in Mooncake; the harness base prefix is 2% of unique content but 27% of re-sent volume. Measured from trace usage fields.

Three quarters of what an agentic model decodes is hidden reasoning — that is what makes the per-turn decode budget



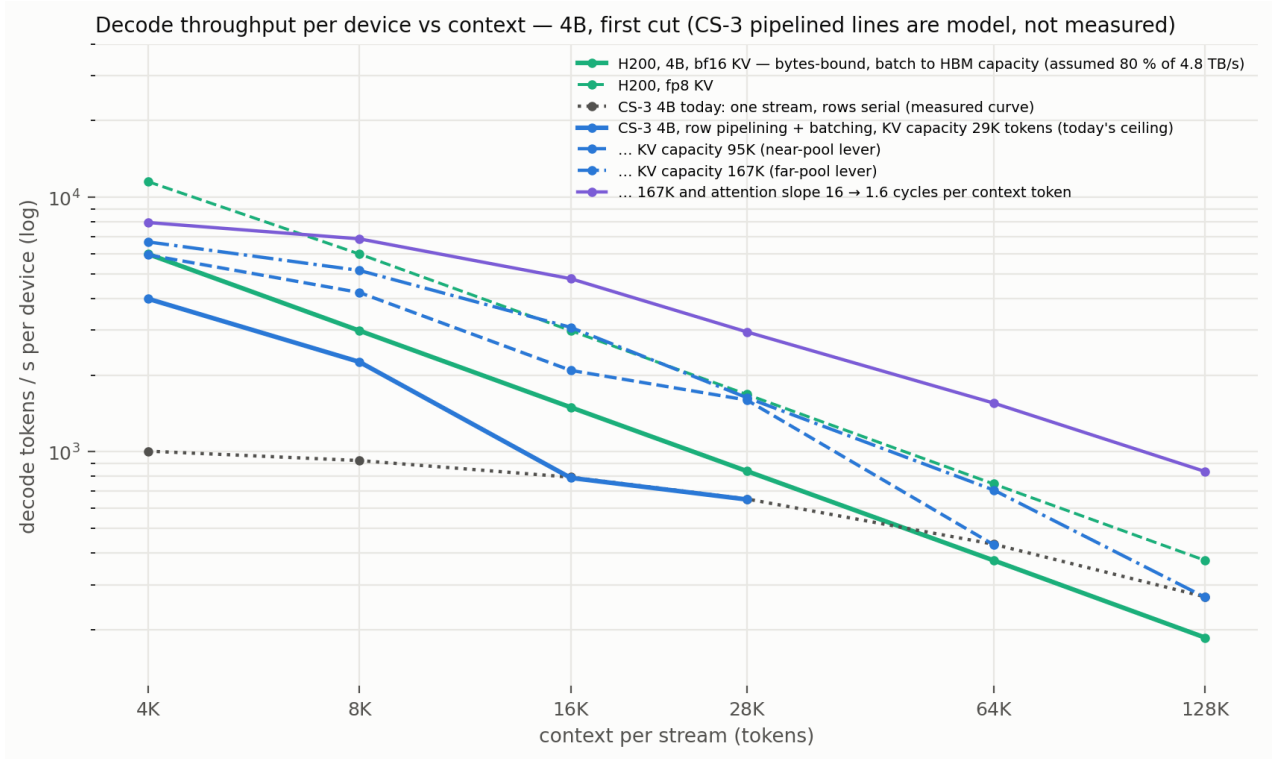
Per-turn decode against per-turn prefill, and an attribution of decoded tokens: hidden reasoning 75%, visible text 5.9%, Edit/Write tool inputs 9.1%, Bash 6.0%, other tools 3.9%. Per turn the median is 5,838 decoded tokens against 10,917 newly prefilled, yet the turn is decode-bound in time on both engines because decode is 10-200x slower per token. Measured token counts; the visible/hidden split is estimated.

Decode cost is linear in context — 681K cycles plus 16.1 per context token — and the device ceiling is 29,184



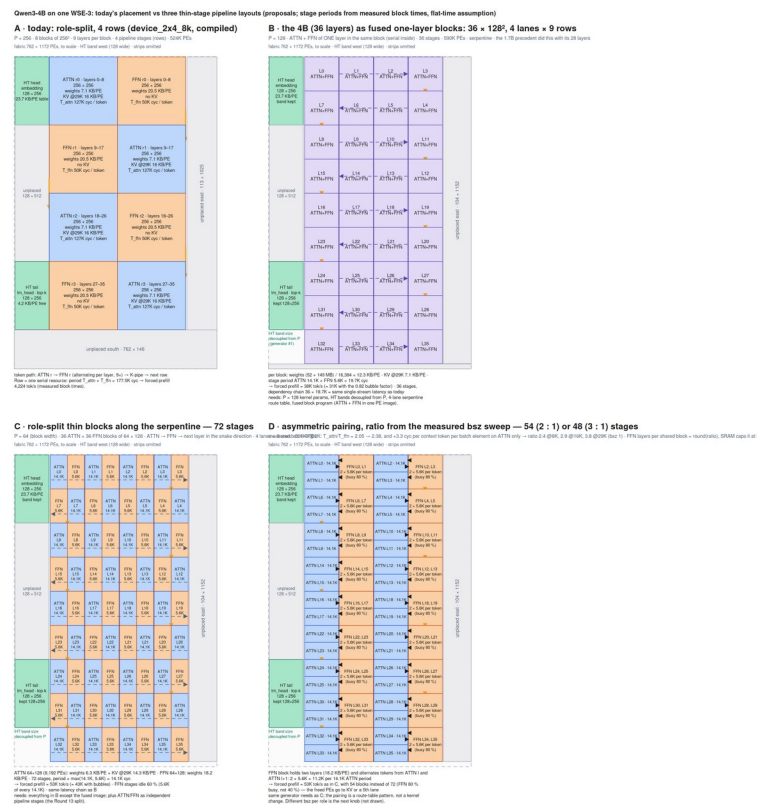
Five device runs of the unmodified route-table 4B decode tree at batch 1 with host-ingested prefixes, 4,096 decode steps each, sweeping MAX_SEQ_LEN from 8,192 to 29,184. From 8K to 29K the step slows 41%; at the ceiling a step is 1,109,064 cycles = 766 tok/s at 0.85 GHz. The next setting, 30,464, fails to link on the appliance. Measured on-device TSC.

Both engines fall as 1/context — and per dollar the GPU keeps a 15-60x margin under every lever set

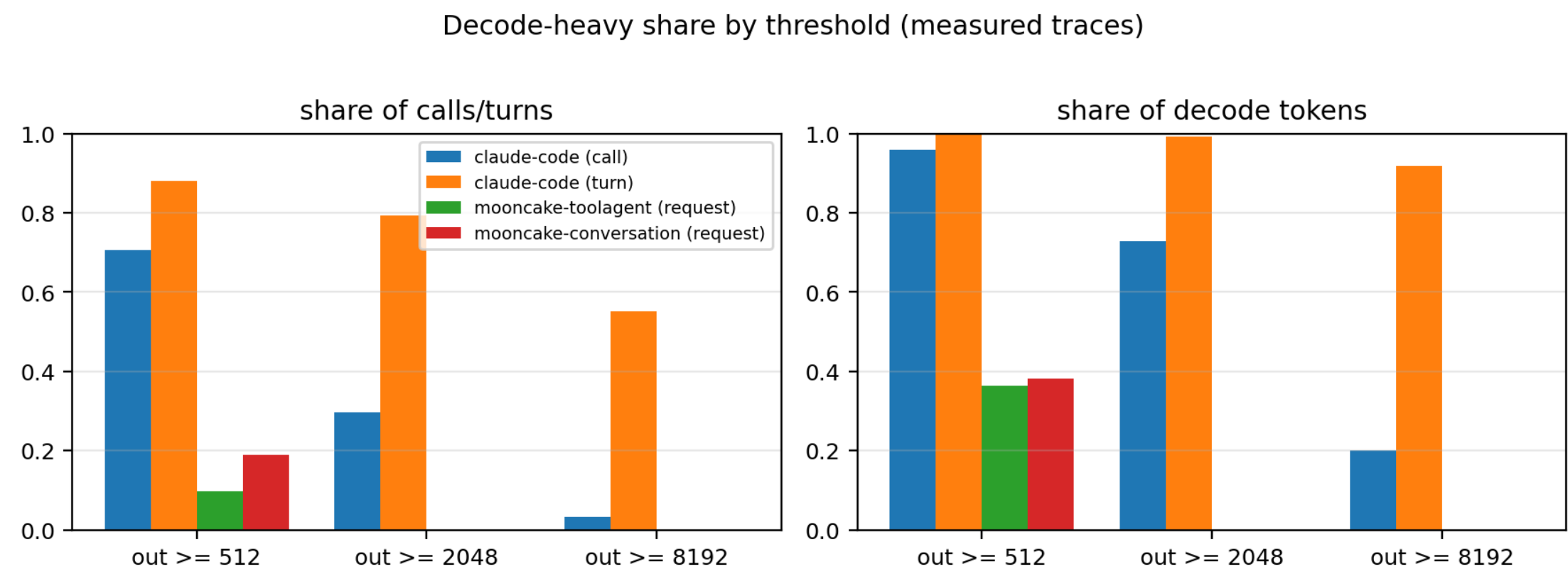


Modelled decode throughput per device against context. The GPU side is bytes-bound (weights plus every stream's KV per step, batched to HBM capacity at 80% of 4.8 TB/s); the CS-3 side uses the measured step laws with row pipelining and batching assumed, and caps the number of streams by on-chip KV capacity. Per device the wafer ends between 0.8x (today) and 3.5x (every lever) an H200. Modelled on measured inputs, not measured.

Four ways to cut the model into pipeline stages — and the three that were never built

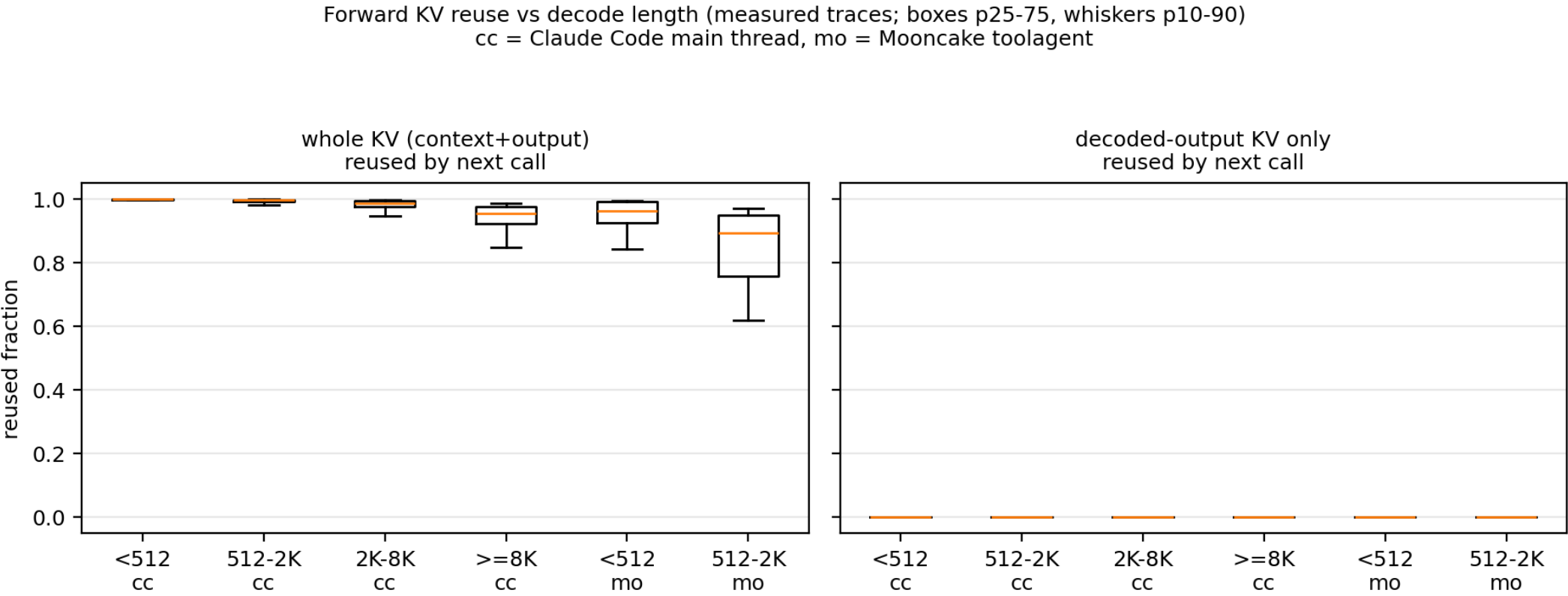


71% of agentic calls decode at least 512 tokens, and those calls carry 96% of all decode tokens



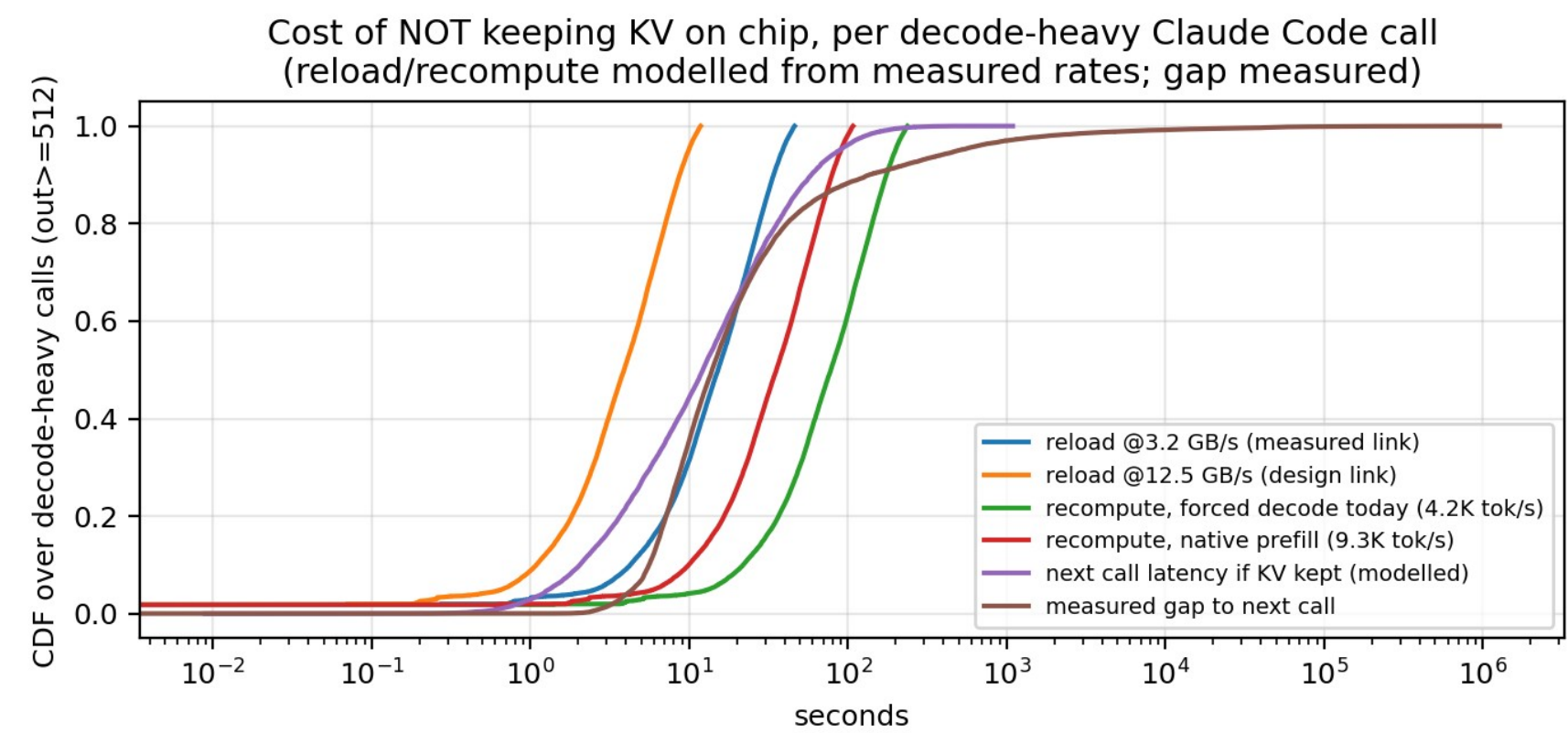
Decode-heavy classes by output tokens, per call and per turn, on both traces. Calls above 8K output are only 3.1% of calls but 55% of turns and 92% of per-turn decode tokens. Mooncake has no request above 2K output at all. Measured on 32,378 de-duplicated Claude Code main-thread calls and 35,639 Mooncake requests.

A decode-heavy call's KV is re-read: 99% of calls have a successor re-reading at least 95% of it



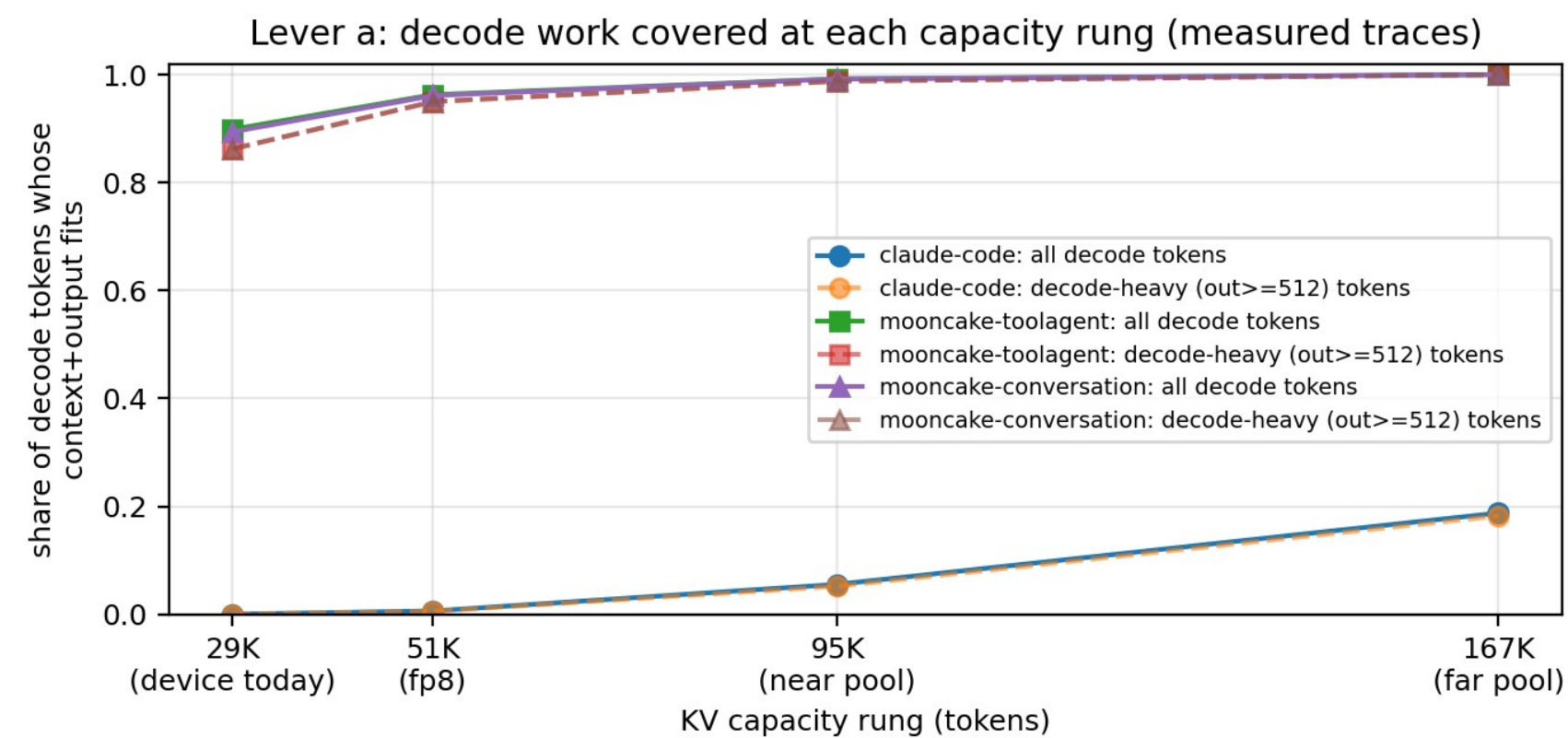
Forward KV reuse by the next call, conditioned on decode length. The median reuse fraction is 0.999 for calls under 512 output tokens and still 0.954 for calls above 8K, with a median gap of 10-18 s. On Mooncake only 37% of decode-heavy requests ever get a next turn, at a median gap of 159 s. Measured trace replay.

Not keeping the context costs a median 13.9 s reload — 61% of the next call's latency



Value of keeping KV on chip per call: reload time avoided, recompute time avoided, and the idle gap it has to fit inside. At the measured 3.2 GB/s link a median reload is 13.9 s and 49% of reloads exceed the gap; at the 12.5 GB/s design link it is 3.6 s and 28%. Keeping the decode's own output KV is worth only 0.003-0.13 s per call, because thinking is stripped before the next call. Measured reuse and gaps, modelled at measured link rates.

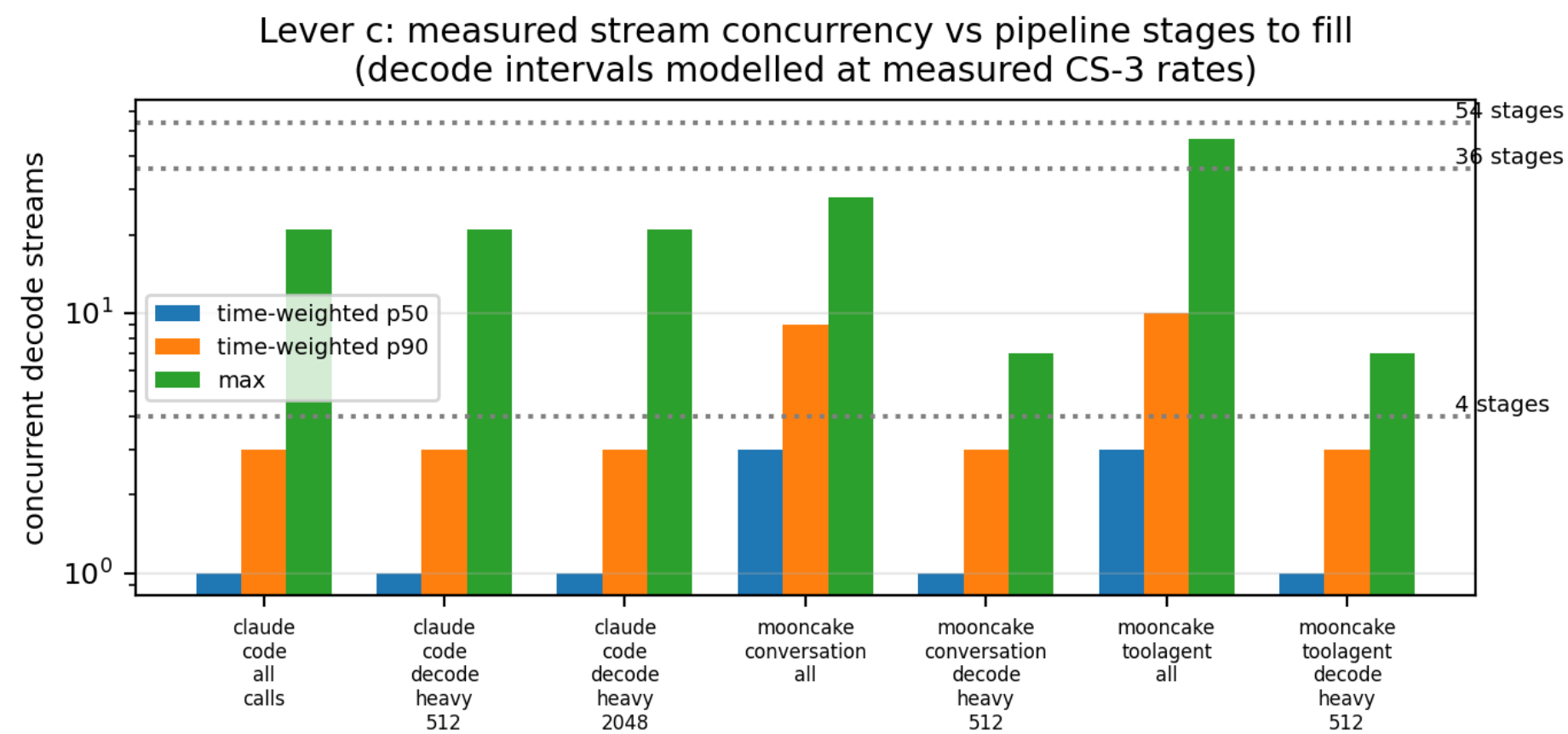
Today's 29K fits 0.04% of agentic calls; even the 167K far pool covers only 19% of decode tokens



Share of decode work covered at each KV-capacity rung — 29K measured on device, 51K with fp8 KV, 95K near pool, 167K far pool. The median agentic call context is 309K tokens (p90 715K), so capacity is binding and still about 2x short of the median. For Mooncake-class traffic the picture inverts: 90% of decode work already fits 29K. Measured contexts against analytical capacity rungs.

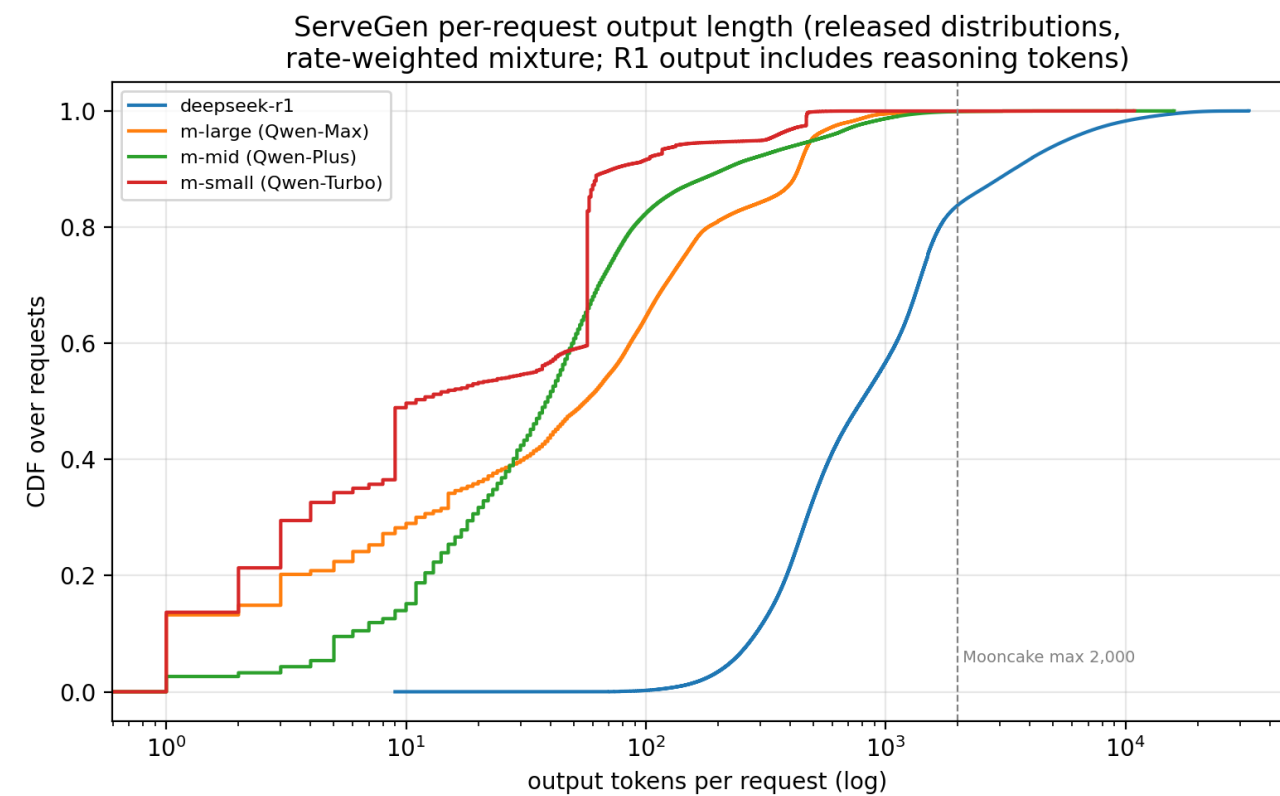
Decode-stream concurrency is 1 at the median on both traces

— filling 36 or 54 stages is an operator question



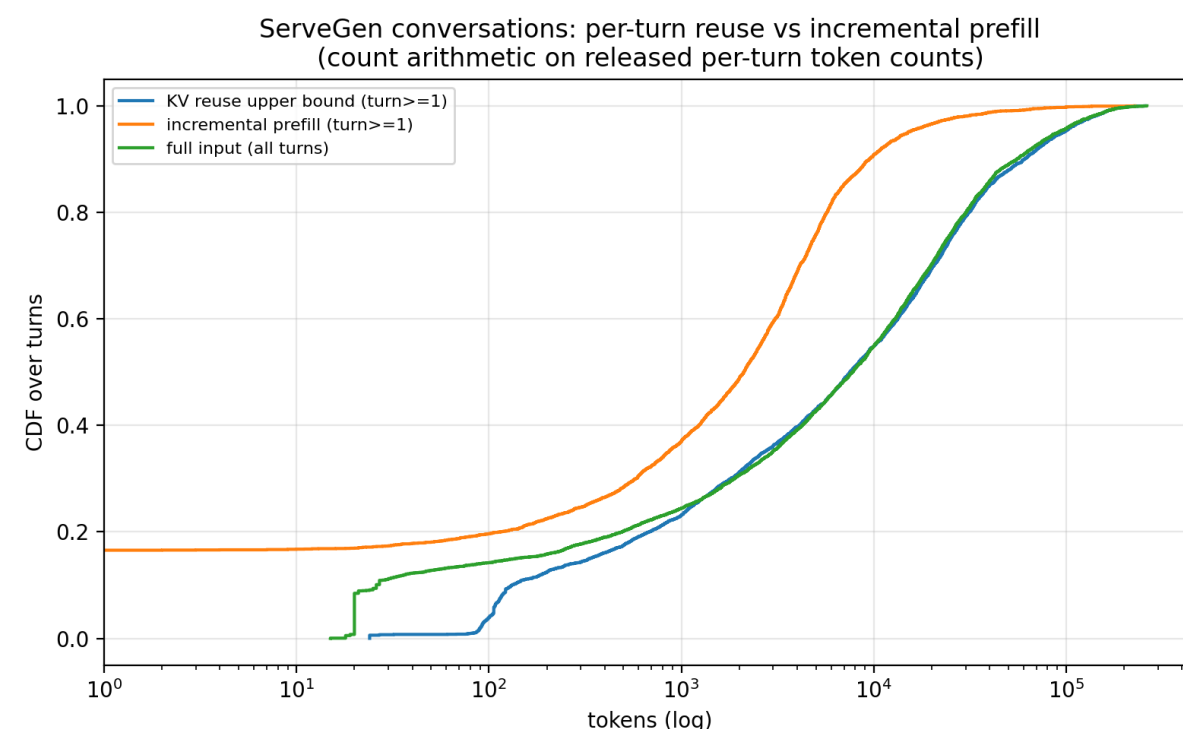
Concurrency of decode-heavy streams over time against the pipeline stage counts under consideration (4, 36, 54). Four stages fill in bursts; 36 and 54 never fill on either trace, with a maximum of 7 concurrent decode-heavy streams on Mooncake. Measured from trace timestamps.

A multi-tenant reasoning platform's per-request output is almost exactly an agentic per-call output



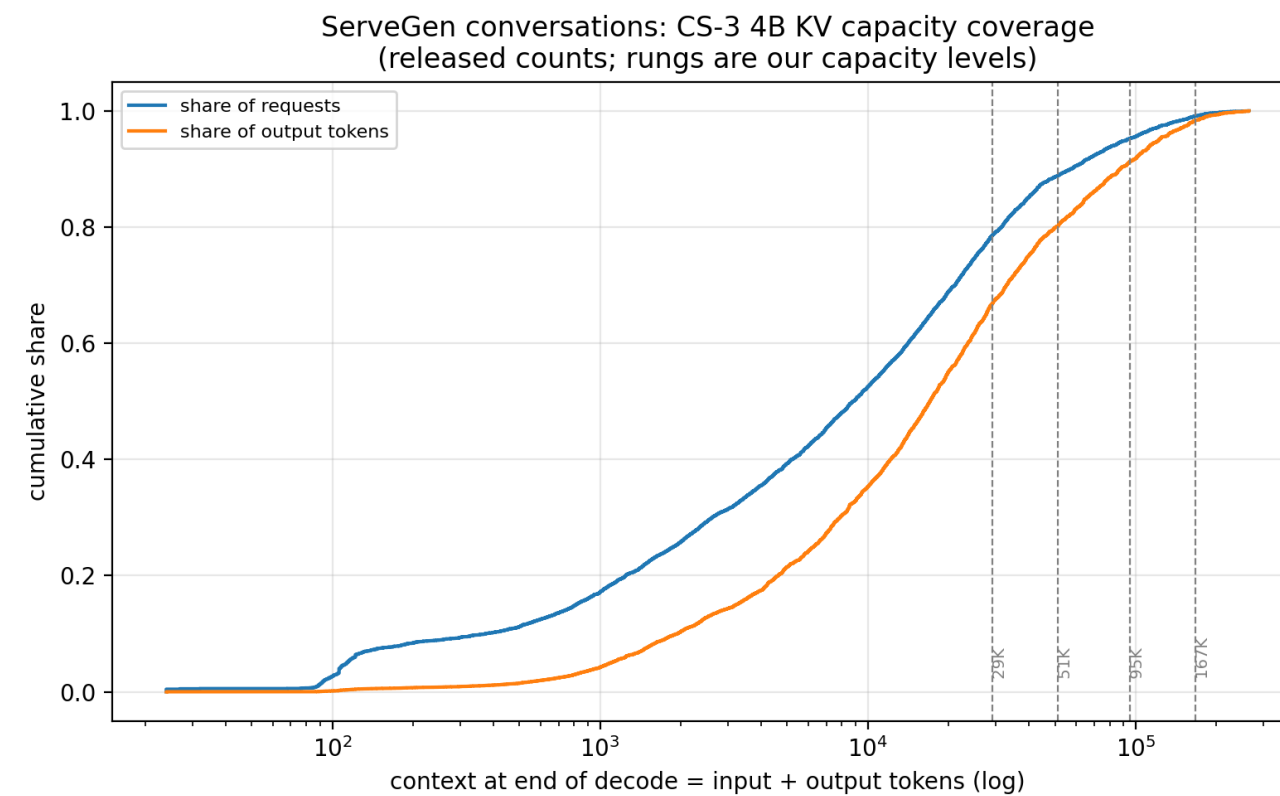
Output-length CDFs from the ServeGen release (Alibaba Cloud Model Studio production, NSDI'26), DeepSeek-R1 including reasoning tokens against the language models. R1's per-request output is p50 792, p90 3,432, max 32,768, with a bimodal reason ratio (0.38 / 0.9, mean 0.70) — statistically the same shape as Claude Code's p50 763 per call. Measured on the released distributions.

This platform re-sends the reasoning: 83.4% of turn pairs grow by at least the previous full output



Per-turn reuse length and incremental prefill across 1,616 ServeGen conversations (5,720 turns, a 24-hour window in February 2025), computed by count arithmetic on input and output token counts without any harness information. Reuse is p50 7,498 tokens (82% token-weighted) and the incremental prefill is p50 2,107. The 83.4% result is the opposite of Claude Code, where thinking is stripped and only ~5% of decoded tokens re-enter context. Measured, with reuse as an upper bound.

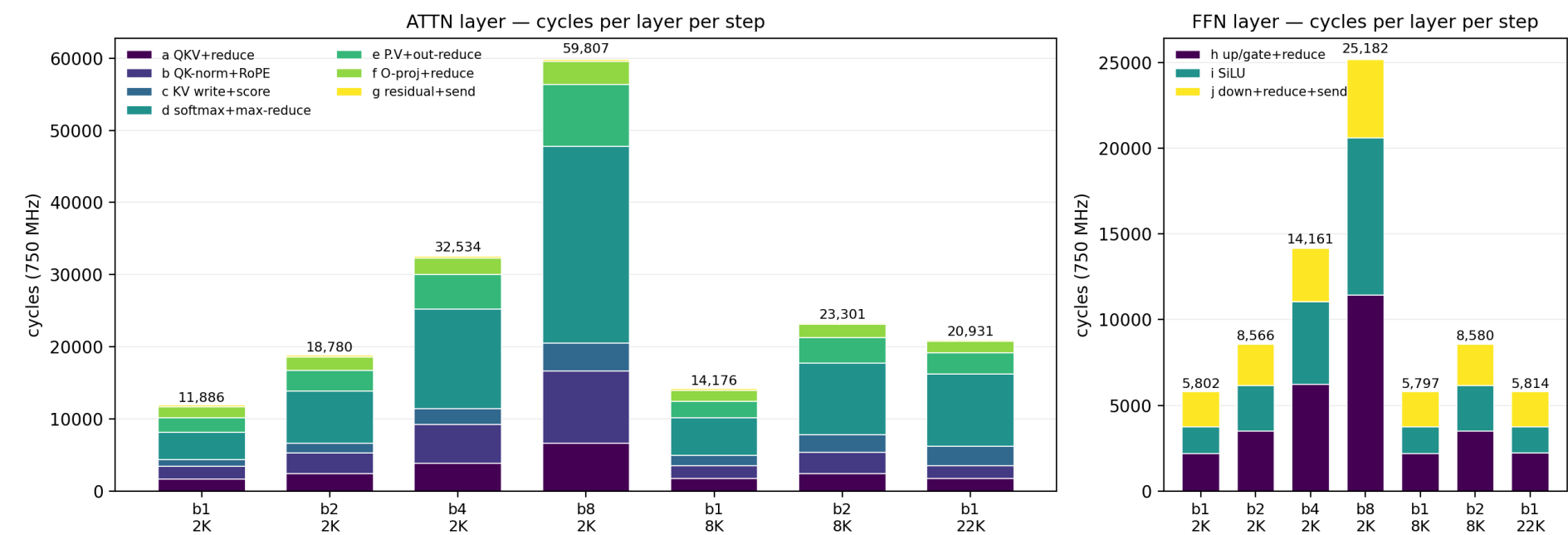
For multi-tenant reasoning traffic 29K already covers two thirds of the decode work



Share of ServeGen requests and of output tokens that fit each KV-capacity rung. 29K holds 78% of requests and 67% of output tokens; 167K holds 99% and 98%. Context sits where Mooncake's does (input p50 7.7K) rather than at Claude Code's 300K, so the 29K and 51K rungs are far more useful on multi-tenant reasoning traffic than on our own trace. Measured on the released conversation file.

Softmax is the largest phase everywhere and carries 69% of the whole context slope

Qwen3-4B decode: phase breakdown inside one ATTN and one FFN layer (CS-3, row 0, steady state)



Phase-level breakdown inside one ATTN and one FFN layer against batch and context, stamped by an on-device TSC probe on today's 256-square blocks across seven configurations. Softmax carries 0.303 of the 0.439 cycles per context token per layer and 49% of the batch slope; the three KV-walking phases together give 15.8 cycles per context token per step over 36 layers, independently reproducing the context sweep's 16.07. The FFN layer is flat in context (5,802 / 5,797 / 5,814 cycles at 2K / 8K / 23K). Measured device TSC.

The three ways to cut a layer into stages: which phases go to which stage



The one-layer testbed's three cuts drawn as full 256-square blocks on today's footprint, with the real hand-offs and the periods derived from the measured phase sums: cut (i) ATTN | FFN (2 stages), cut (ii) A1 | A2 | FFN (3), cut (iii) A1 | scan | O-proj | F1 | F2 (5). Topology sketch; the derived periods come from measured phases, the areas do not mean anything.

Comparing cuts needs equal active area — and the strip heights drawn here were wrong in five of six setups

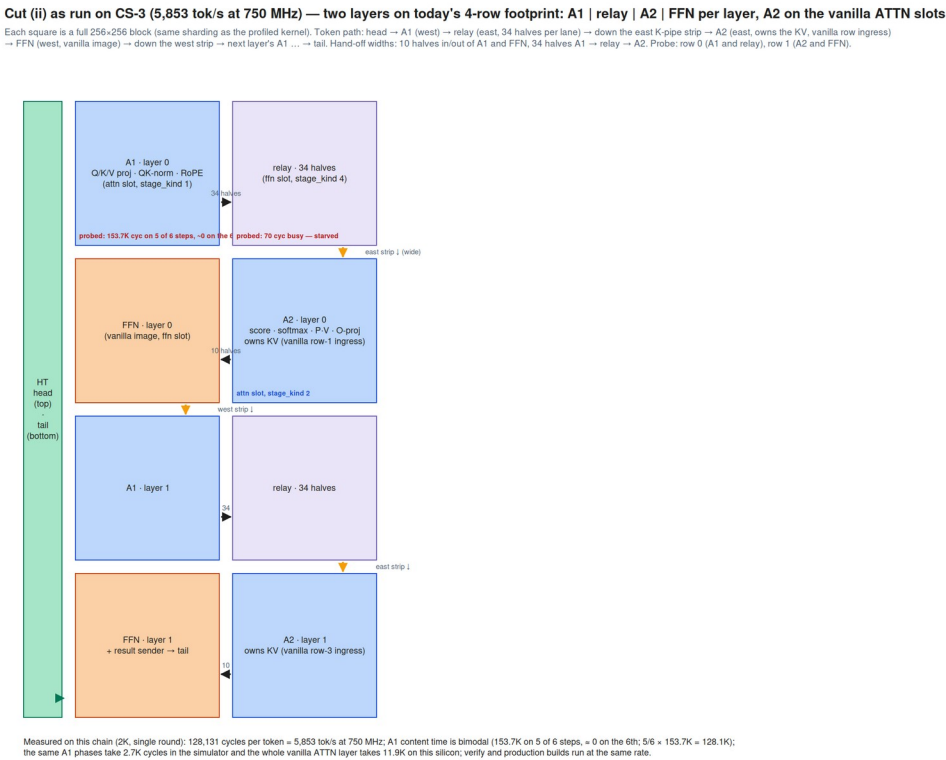
Equal-area one-layer testbed on today's row footprint — ATTN-role strips in the west square, FFN-role strips in the east square (Round 47)

Today a row is ATTN 256² | FFN 256² side by side (512 × 256). The cuts keep that: each stage is a 256-wide strip of its square; tokens flow top → bottom in the west square, then east to the FFN strips (the existing east-west hand-off), top → bottom there, then on to the next layer / tail. Every cut gets the same total area per budget. Strip heights are illustrative; cycles are Round 45's 256²-block phase sums at 2K, bez 1 — unknown at strip heights, which is what the testbed measures.



The equal-area testbed on today's side-by-side row, attention-role strips west and FFN-role strips east, with each cut drawn at budget A (a full row) and budget B (a layer's real deployment share, about 57 rows). The heights in this drawing are illustrative and Round 52 checked all six setups: only A-(i) is feasible as drawn. Every other setup violates height admissibility (a strip needs dim % h = 0, and KV holders and FFN strips need a power of two) or overflows SRAM. Design figure, superseded in detail.

The chain that actually ran on silicon for cut (ii) — and where the A1 stall showed up

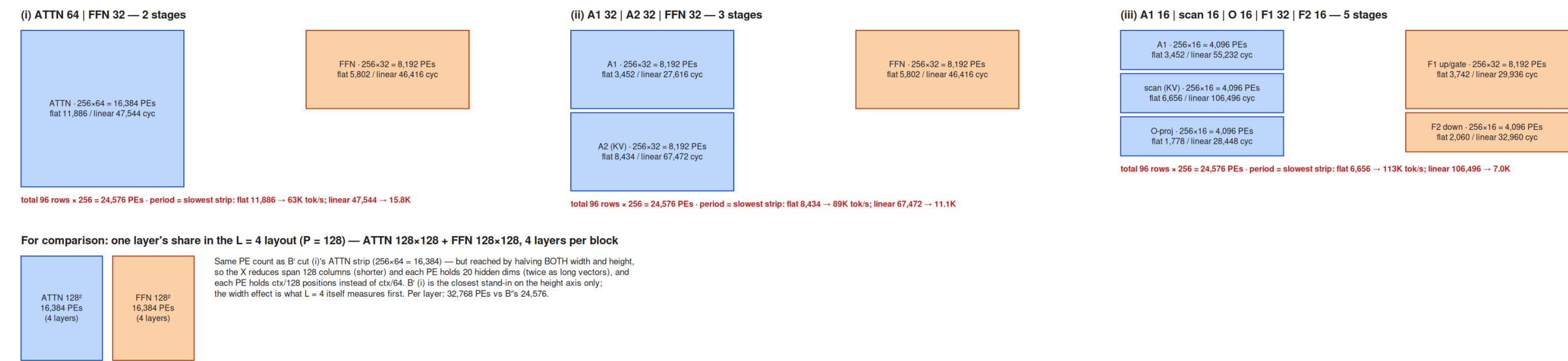


Cut (ii) as run on CS-3: today's four-row, two-square footprint with two layers laid out as A1 -> relay -> A2 -> FFN, the probe on row 0 and row 1. Faithful to the run's own placement plan. On device the chain is functionally correct — the restated oracle gate passes with zero defects over 1,000 steps — but it runs at 128,131 cycles per token against a derived 8,434, because A1 stalls on five of every six steps. Measured on device.

The equal-area testbed as drawn — superseded before it ever ran

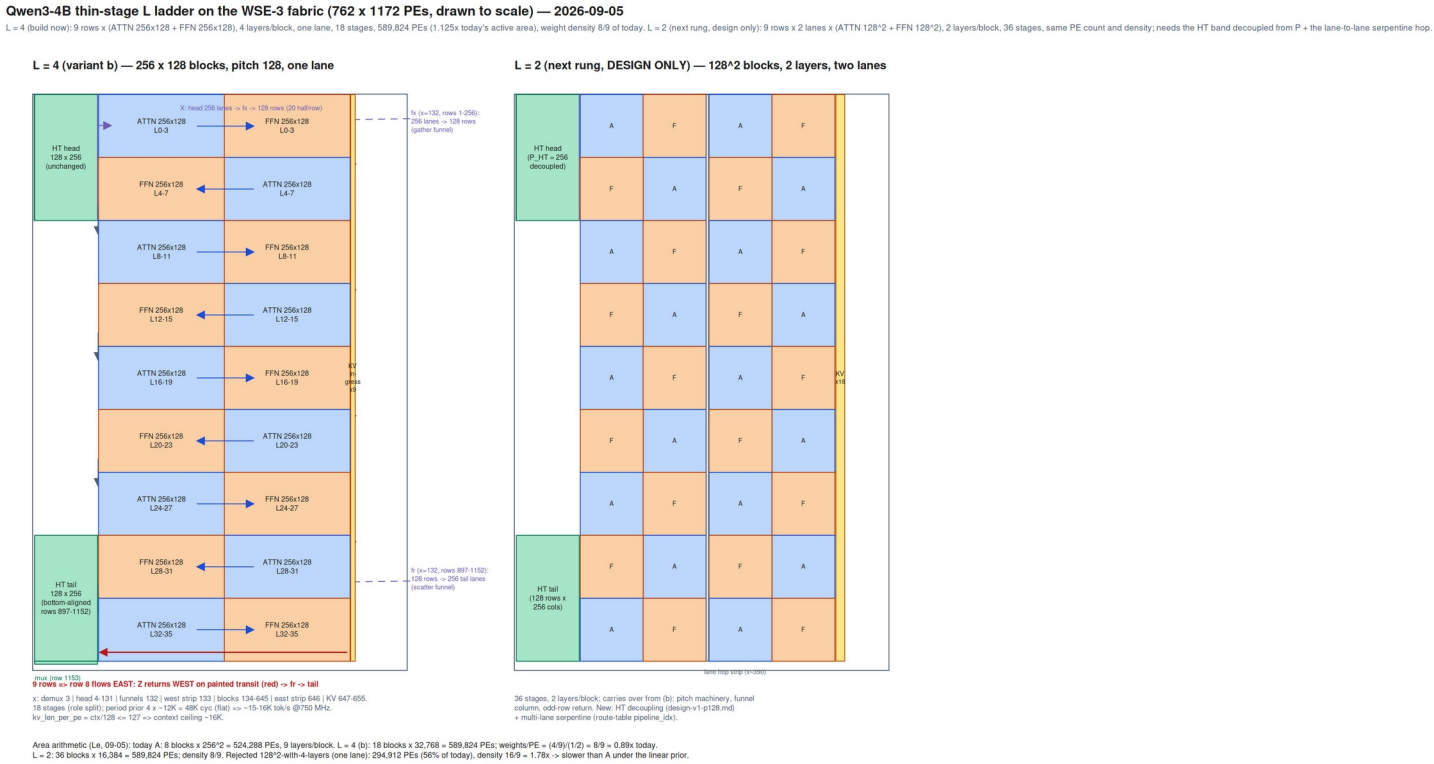
B' side by side — every cut gets the same 96 rows × 256 columns (24,576 PEs); rows are given to the slow stages so the periods come out alike

West column = attention-role strips, east column = FFN-role strips (as in today's row). Inside each strip: PEs, and the strip's expected period under the two priors — flat (Round 45 phase sum, i.e. per-layer time does not change with strip size) and linear (time scales with 65,536 / PEs). The measurement decides where reality sits and which cut balances best. Heights are the subagent's confirmed B' heights (Round 52).



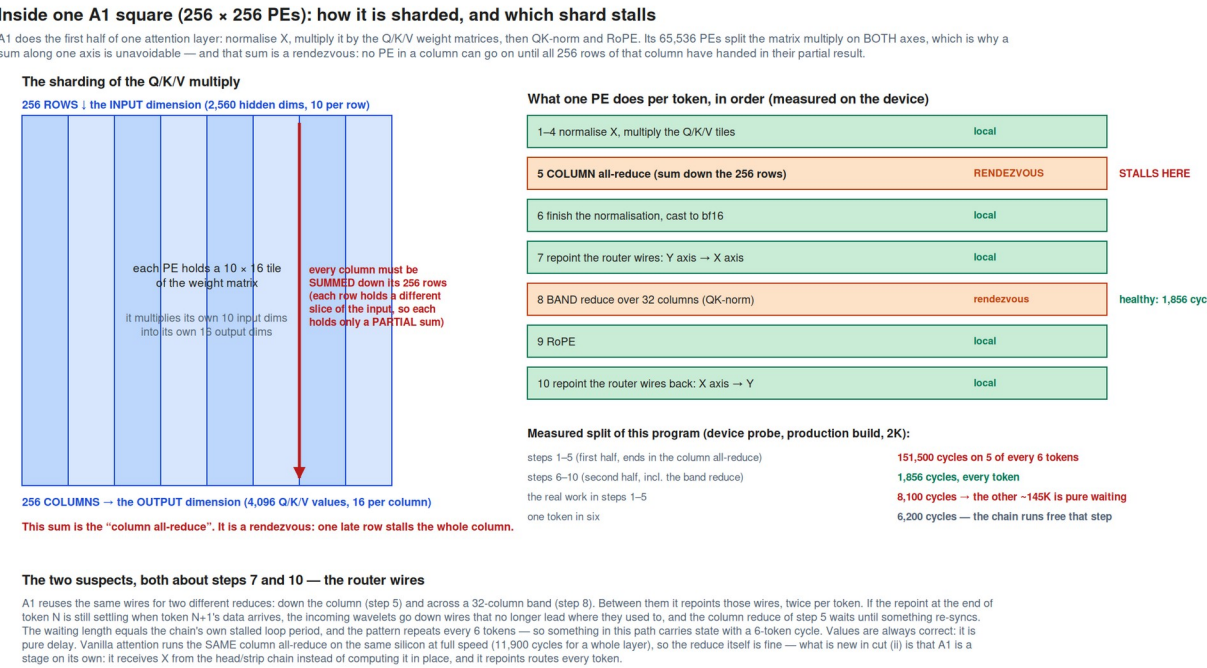
The three cuts side by side at a common 96-row budget with strip heights chosen to balance the stages under the linear prior, plus a 128-square pair drawn for comparison. Round 63 superseded the comparison pair: a 128-square block is a quarter of today's block and is therefore the L = 2 rung, while the equal-area L = 4 is 256 wide by 128 tall. The 96-row budget itself was then overtaken by compiled SRAM. Design figure, superseded.

The thin-stage layouts drawn to fabric scale: L = 4 on one lane, and the two-lane rung that stayed on paper



Left panel: L = 4 as nine rows of ATTN 256x128 plus FFN 256x128 on one lane, with the funnel column and the odd-row result return — this is the layout that was built, gated and measured at 48,872 cycles per token. Right panel: an L = 2 design at 128-square blocks on two lanes, 36 stages, which needs the HT band decoupled from P and a multi-lane serpentine and was never built. Design figure for a measured layout.

Why A1 stalls: a 256-row column rendezvous where one late row stops the whole column

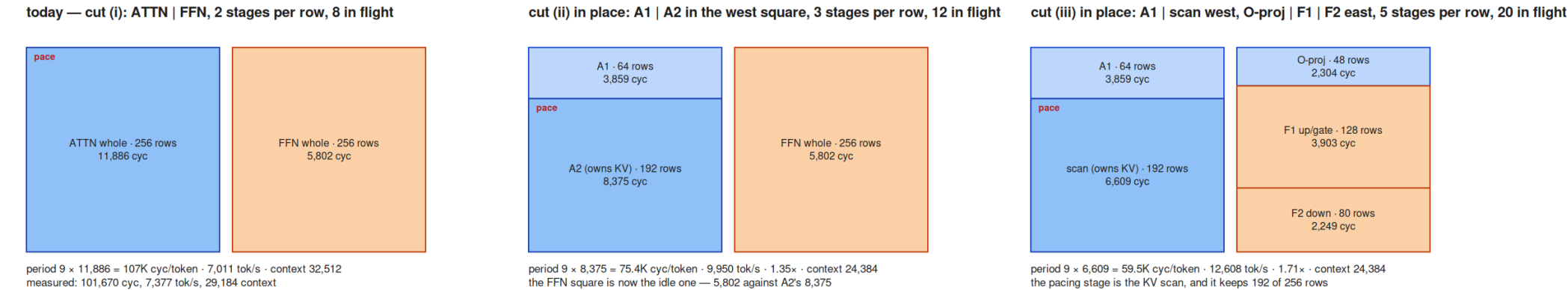


Inside one A1 square: the Q/K/V weight matrix is sharded on both axes — the input dimension down the 256 rows, the output dimension across the 256 columns — so every PE ends with a partial result and the column all-reduce is a rendezvous. The device probe split A1's per-token program in two and localised the stall to the first half (RMSNorm, the three matvecs, the column all-reduce): 151,500 cycles on five of every six tokens against 8,100 cycles of real work, while the second half is steady at 1,856. Measured device probe.

The operation cuts belong inside today's allocation, not one square per stage

The operation cuts placed inside today's allocation — same two 256×256 squares per row, same 9 layers per block, only the squares cut into strips

Le's point: to compare cuts you must hold the area fixed. Today one row is [ATTN 256×256 | FFN 256×256] and carries 9 layers; a finer cut must live in that same footprint, not take a whole square per stage. Cutting a square horizontally also keeps the hand-off local — no relay square, no cross-row wide transfer. Strip heights below are chosen to balance the stages using the measured phase times (2K, 256², Round 45) and the measured height-scaling curve (Round 77: ATTN 0.99× at 128 rows, 1.12× at 64, 1.47× at 32; FFN 1.04× / 1.11× / 1.73×). Only the KV-holding stage needs height: context cap = 127 × its rows.



Why this beats thinning L
Thinning L (fewer layers per block, shorter blocks) multiplies stages by making every block shorter, so the KV a PE must hold — context ÷ block height — grows and the context ceiling collapses: L = 4 gives 2.08× at 16K context, L = 1 gives 5.72× at 4K. Cutting the operations instead multiplies stages while leaving the ONE stage that holds KV its full height: cut (iii) above keeps 192 rows for the scan, so the context cap stays near today's. 1.71× at ~24K context is a different trade from 2.08× at 16K — and the two axes compose, since a thinner L can be cut this way as well.

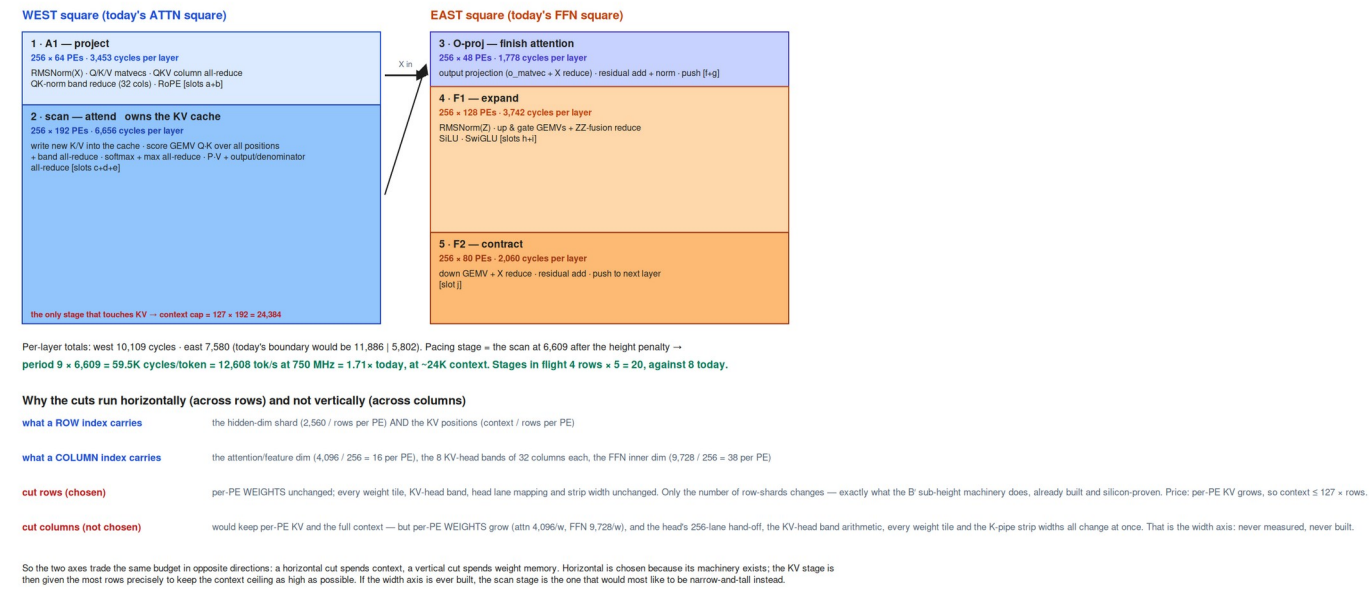
Predicted, not measured: the phase times are measured at 256² and the height curve is measured for whole ATTN/FFN layers; per-phase height scaling is the thing the equal-area testbed measures.

The same two 256x256 squares per row and the same nine layers per block as today, with the squares cut into horizontal strips whose heights balance the stages: today's cut (i), then cut (ii) in place, then cut (iii) in place. Predicted periods use the measured 256-square phase times and the measured height-scaling curve — cut (ii) 1.35x and cut (iii) 1.71x today. Predicted, not measured; neither in-place cut was built.

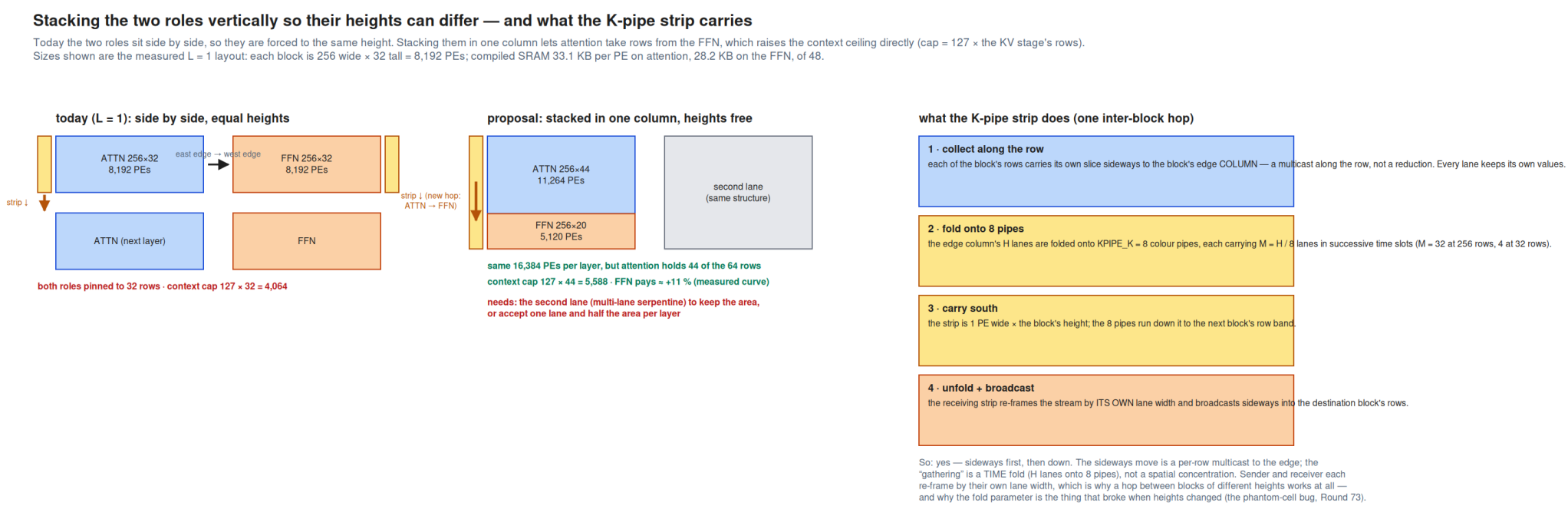
Cut (iii) in place: five stages, and only one of them touches the KV cache

Cut (iii) inside today's allocation: one row = two 256x256 squares, 9 layers per block — the squares cut into horizontal strips

Same footprint and same layer count as today; only the two squares are divided. Each strip below names the computations it owns (phase slots from the measured phase-TSC profile) and its measured cost per layer at 2K on a full 256² square. Strip heights are chosen so the stages balance under the measured height-scaling curve. Data flows west → east, then down the serpentine to the next row.

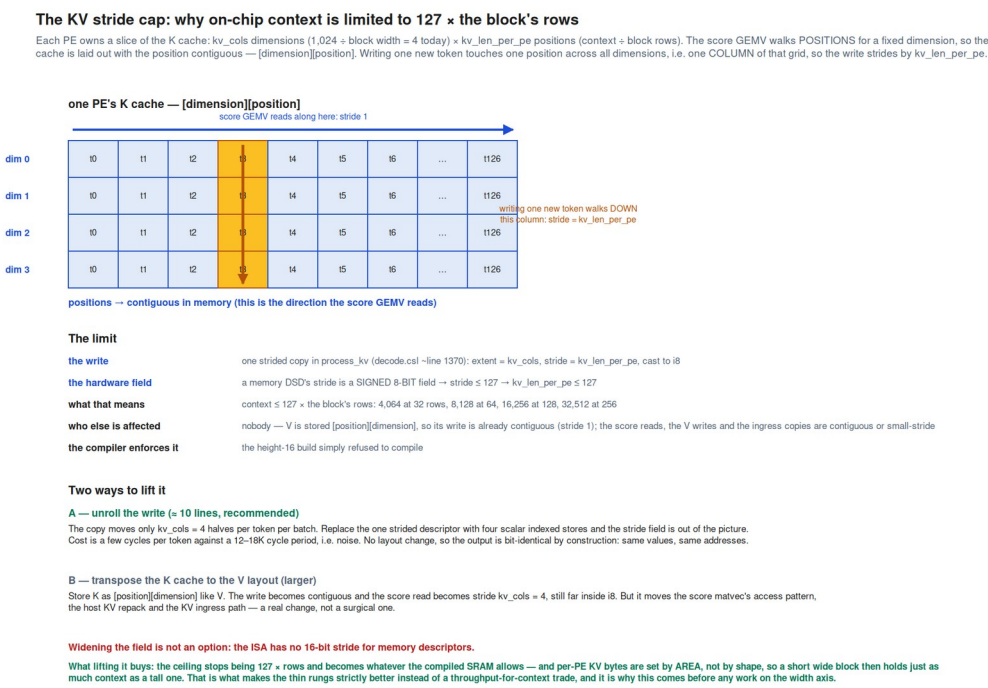


Stacking the two roles vertically so their heights can differ, and what the K-pipe strip carries



Today's side-by-side L = 1 row against the stacked proposal, plus the four steps of the inter-row K-pipe: collect along each row to the edge column, fold the lanes onto eight colour pipes as a time fold, carry south down a one-PE-wide strip, then unfold and broadcast into the next block. Sender and receiver re-frame independently, which is what makes a hop between blocks of different heights possible at all. Stacking was then built and measured: 12,202 cycles per token on a three-layer rig, with attention's own compute unchanged at -1.3%.

One 8-bit stride field capped on-chip context at 127 positions per row — and ten lines removed it



One PE's K cache laid out as [dimension][position]: the score GEMV walks every position for a fixed dimension, so positions must be contiguous, which makes the write of one new token a strided copy with stride = positions per PE. A memory descriptor's stride is a signed 8-bit hardware field, so positions per PE is capped at 127 and context at 127 x block rows. Option A unrolls the write into four scalar stores; option B transposes the cache to the V layout. Option A was implemented and gated byte-identical on 1,000 device records.

Prefill measures 10,544-11,143 tok/s; in-kernel batching saturates at ~2,370

First CS-3 measurements of the 4B: native prefill at two chunk sizes, and decode at batch 1 through 8. A selection of 5 of 7 measured points — bsz 2 (1,289,187 cyc/step) and bsz 4 (2,102,119) are omitted and both sit on the fitted line. Measured on-device TSC; tok/s converted at 0.85 GHz.

RUN	CYCLES	TOK/S @0.85 GHZ
P1 — prefill, 1 x 8,192 tokens	76,282 / token	11,143
P2 — prefill, 4 x 2,048 tokens	80,615 / token	10,544
Decode, bsz 1 (TOP_K 20)	785,826 / step	1,082
Decode, bsz 8	3,300,460 / step	2,061
Fitted law 427K + 359K x bsz	359K per element	~2,370 asymptote

Per-layer time barely moves with region size — the smallest region that links is the fastest

MeshRT Qwen3.5-9B layer_block benchmark on CS-3 at batch 1 and 4K context, eight regions, 12 device jobs. A selection of 5 of the 8 measured regions: 128x512 (+1.1%), 256x512 (+5.3%) and 512x256 (+17%) are omitted, and 16,384-PE regions do not link at all. Measured device TSC.

REGION (W X H)	PES	FULL CYC/STEP	VS 256-SQUARE
128 x 256	32,768	15,647	-0.3%
256 x 128	32,768	15,718	+6.6%
256 x 256 (published)	65,536	14,719	0
512 x 128	65,536	16,923	+20%
512 x 512	262,144	17,787	+22%

A block row is one serial resource: ATTN and FFN alternate per layer, so they cannot hold different tokens

One PE per block stamps its own TSC at the per-layer boundaries; bursts leave through new routing-only collector streams, so no existing data path is touched. Row 0, steady state over the last 512 of 4,096 steps at about 8K context. Measured device TSC.

QUANTITY (ROW 0)	CYCLES	WHAT IT MEANS
T_attn — 9 ATTN layers	127,301	14,145 per layer, 72% of a layer
T_ffn — 9 FFN layers	50,238	5,582 per layer
Waits: ATTN on FFN / FFN on ATTN	44,981 / 113,313	The roles alternate every layer
Row-0 step period	808,995	Row busy only 21.9% of the step
T_attn + T_ffn = row period	177,539	4,788 tok/s forced, at 0.85 GHz

Decode cost is linear in context, and the device ceiling is 29,184 — the next setting will not link

Five device runs of the unmodified route-table decode tree at batch 1 with host-ingested prefixes, 4,096 decode steps each. The fit over the five points is 681K + 16.1 cycles per context token per step; the sixth setting tried, 30,464, fails to link on the appliance.

Measured on-device TSC.

MAX_SEQ_LEN	CONTEXT DURING DECODE	CYCLES/STEP	TOK/S @0.85 GHZ
8,192 (reference)	4K -> 8K	785,826	1,082
16,384	12K -> 16K	889,547	956
23,040	18.7K -> 22.8K	1,029,112	826
28,160	23.8K -> 27.9K	1,096,070	776
29,184	24.8K -> 28.9K	1,109,064	766

The ATTN:FFN ratio never inverts — ATTN grows faster with batch and is the only role that grows with context

Block-TSC probe rerun at batch 1, 2, 4 and 8 with context held equal at 2K, five device jobs, tokens byte-identical to unprobed runs. Fits: $T_{\text{attn}} = 45.2\text{K} + 61.7\text{K} \times b$, $T_{\text{ffn}} = 27.2\text{K} + 24.9\text{K} \times b$ per nine-layer row. Measured device TSC; rates at 0.85 GHz.

BSZ	T_ATTN	T_FFN	RATIO	ROWS-AS-STAGES TOK/S
1	106,645	51,964	2.05	5,358
2	168,577	77,056	2.19	6,919
4	292,757	127,240	2.30	8,093
8	538,793	226,604	2.38	8,882

Capacity is binding, keeping KV pays per hit, and pipelining is an operator question

The decoder-only lever evaluation's verdict, from replaying 32,378 de-duplicated Claude Code calls and 35,639 Mooncake requests against measured CS-3 rates and modelled keep/reload/recompute economics.

LEVER	WORK IT TOUCHES	VALUE PER HIT	VERDICT
a — capacity 29K -> 167K	Agentic decode tokens 0.01% -> 19%; Mooncake 90% -> 100%	Precondition for lever b	Binding; ~2x short of the median agentic context
b — keep KV across calls	99% of agentic calls, >=95% reused in 10-18 s; 37% of Mooncake	13.9 s = 61% of next-call latency; decoded-KV part <=0.13 s	High per hit, gated by a; fast recompute substitutes
c — pipelining / batching	Concurrency p50 1 on both traces, max 7-47	Fills 4 stages in bursts; 36 or 54 never on-trace	Multi-tenant / operator question only

58,581 of ~105K trace rows were duplicates — the agentic decode budget is 5.8K tokens per turn, not 14.4K

One API call is logged as several assistant records, one per streamed content block, each repeating the same message id and usage. A selection of 5 of the 13 corrected quantities; also corrected were calls per turn (~19 -> ~12) and per-turn new prefill (24,691 -> 10,917). Every CS-3 measurement and every ratio was unaffected.

QUANTITY	BEFORE	AFTER
Claude Code API calls	102,517	46,650
Intra-turn gap p50 / p90	3.3 s / 22 s	10.9 s / 50 s
Per-turn decode tokens p50 / p90	14,373 / 66,685	5,838 / 23,546
Hidden-reasoning share of decode	89%	75%
Median turn on CS-3 (29K curve)	26.0 s	10.5 s

Multi-tenant reasoning traffic is decode-heavy but not deep-context

Three-way comparison of the traces the argument rests on. A selection of 5 of 7 axes: incremental prefill (p50 2,107 / 1.5K / new input dominates) and idle gaps (inter-turn 308 s / intra-turn 10.9 s / 3.0 s) are omitted. Measured on each release; ServeGen reuse is an upper bound and its concurrency is derived.

AXIS	SERVEGEN R1	CLAUDE CODE	MOONCAKE
Per-request output	p50 792, p90 3,432	p50 763 per call; 5.8K per turn	p50 350, max 2,000
Reuse length	p50 7,498 (upper bound)	context p50 ~302K, 98% re-sent	37-57% block reuse
Output re-enters next input	Yes — 83% fully	~5% — thinking stripped	0% (toolagent)
Fits 29K / 167K	78% / 99% of requests	0.04% / 19% of decode tokens	~90% fits 29K
Decode-stream concurrency	26 mean / 68 peak (derived)	1 user	3.4-6.7 req/s arrivals

Two wafers of today's layout do not move the needle — only two wafers with the far-pool levers reach half the calls

Capacity rungs against how much of the agentic workload each covers. Capacity figures are measured (29K) or analytical (the rest); coverage is measured on the de-duplicated Claude Code trace.

KV CAPACITY	WHAT IT IS	CALLS THAT FIT	DECODE TOKENS
29,184	Today, measured on device	0.1%	0.0%
53-58K	Two wafers, today's layer split	3.0%	1.3%
95K	Near-pool lever	10.0%	6.5%
167K	Far-pool lever (score wall)	24.5%	21.6%
334K	Two wafers x far pool	54.7%	52.0%

For dense-attention models the per-wafer resident context stays 24-32K whatever the model size

Resident KV capacity = eta x (N x 44 GB - weights) / KV bytes per token, with eta ~ 0.13 calibrated on two independent measured anchors. A selection of 5 of 8 rows: Qwen3-8B (24K), Qwen3-30B-A3B (94K on 3 wafers) and GPT-OSS-120B (1.0M on 8 wafers) are omitted. Analytical, on measured anchors.

MODEL	WAFERS	KV/TOKEN	TODAY (ETA 0.13)	PER WAFER
Qwen3-4B bf16	1	144 KB	32K (measured 29K)	32K
Qwen3.5-9B, dense attention	2	144 KB	62K	31K
Qwen3.5-9B, 3:1 hybrid (GDN)	2	36 KB	247K	123K
GPT-OSS-20B MXFP4	4	24 KB	863K (kernel: 900K)	216K
Qwen3-235B-A22B bf16	12	188 KB	39K	3K

Forced prefill runs at exactly one token per row period — there is no pipelining penalty at four stages

Forced mode implemented as a runtime flag plus a baked token list on the HT head; the same compiled image serves both modes. Four device jobs, about 16 minutes of CS-3 time, all gates pass. Measured on-device TSC; rates at 0.85 GHz.

CONTEXT	FREE (SERIAL)	FORCED	CYCLES/TOKEN	MEASURED / PREDICTED
2K	1,263 tok/s	5,674 tok/s	149,811	1.044
8K	1,081 tok/s	5,108 tok/s	166,407	1.032

ATTN and FFN as independent stages: 8,360 tok/s at 2K, byte-identical, 1.47x the four-stage rate

Eight stages, two tokens per row, period = T_attn. The verify build computes top-k every step for the byte gate; the production build runs the logits once per round and streams the head's tokens. Production equals verify to within 0.03%, so the tail was not binding at eight stages. Measured on-device TSC; rates at 0.85 GHz.

BUILD	CONTEXT	CYCLES/TOKEN	TOK/S @0.85 GHZ	GATE
Verify (tail computes top-k)	2K	101,723	8,356	Byte-identical to free
Production (tail skips logits)	2K	101,670	8,360	Final record equal
Verify	8K	118,550	7,170	Byte-identical to free
Production	8K	118,554	7,170	Final record equal

Where a layer's cycles go: softmax dominates and grows fastest with both context and batch

Phase-TSC probe on today's 256-square blocks, seven device runs, all four byte-identity gates pass. Columns are a selection from the full ten-phase table (see F21); rows are 5 of the 7 runs — bsz 2 at 2K (softmax 7,185) and bsz 4 at 2K (13,784) are omitted. Measured device TSC, cycles per layer per step, row 0.

RUN (BSZ, CONTEXT)	SOFTMAX	ATTN LAYER	FFN LAYER	SOFTMAX SHARE
bsz 1, 2K	3,808	11,886	5,802	32%
bsz 8, 2K	27,241	59,807	25,182	46%
bsz 1, 8K	5,211	14,176	5,797	37%
bsz 2, 8K	9,938	23,301	8,580	43%
bsz 1, 23K	9,984	20,931	5,814	48%

Cutting a layer five ways would raise the stage rate 1.3-1.8x — derived from measured phases, not measured

Each cut's pacing stage is the longest contiguous run of measured phase slots; throughput is 0.85e9 divided by that. Derived, under three assumptions the one-layer testbed exists to replace: each cut stage becomes its own PE region, per-phase time is unchanged, hand-offs are free, and there are enough PEs.

RUN	ATTN+FFN (2/LAYER)	A1 A2 FFN (3)	5-WAY CUT
bsz 1, 2K	11,886 cyc -> 71.5K tok/s	8,434 -> 100.8K	6,656 -> 127.7K
bsz 8, 2K	59,807 -> 14.2K x b	43,171 -> 19.7K	39,740 -> 21.4K
bsz 1, 8K	14,176 -> 60.0K	10,669 -> 79.7K	8,894 -> 95.6K
bsz 2, 8K	23,301 -> 36.5K x b	17,974 -> 47.3K	15,929 -> 53.4K
bsz 1, 23K	20,931 -> 40.6K	17,445 -> 48.7K	15,659 -> 54.3K

At full block size the phase-sum model predicts the measured stage period to 0.2%

Cut (i) — ATTN | FFN with one layer per block on today's full squares — needed no kernel change: at one layer per block the serial kernel is already a two-stage pipeline. Six device jobs, all gates pass, verify records byte-identical to same-tree free references. Measured on-device TSC; rates at 0.85 GHz.

CONTEXT	CYCLES/TOKEN	TOK/S @0.85 GHZ	DERIVED	MEASURED / DERIVED
2K	11,866	71,633	11,886 cyc	1.002
8K	13,705	62,021	14,176 cyc	1.034

Exact softmax fusion is byte-identical and 8-14% slower — the lever is DSD overhead, not fusion

Folding the sum and cast passes into the exp pass removes exactly what it targets and loses two to three times that inside the exp loop. Ten device jobs, all gates pass; every exact variant is byte-identical on device. Measured device TSC, softmax cycles per layer per step.

CONFIG	BASELINE	FUSED (A2)	CHANGE
2K, bsz 1	4,103	4,581	+11.7%
8K, bsz 1	5,502	5,958	+8.3%
23K, bsz 1	10,246	11,030	+7.6%
23K, bsz 1, variant (c)	10,246	10,781	+5.2%
2K, bsz 8	29,342	33,360	+13.7%

Compiled numbers, not tables: height 16 does not compile, and height 32 fits with 16-20 KB per PE to spare

Real-geometry compile-only builds of the 2K cut (i) geometry at heights 16 and 32 with cs-readelf memory reports, run after Le's rule that only compiled SRAM numbers count. Compiled (link-time), per PE of a 48 KB bank, uniform within regions.

REGION	KB/PE AT HEIGHT 32	HEADROOM
ATTN strip (256 x 32)	32.2	15.8 KB
FFN strip (256 x 32)	27.8	20.2 KB
HT tail (binding region)	42.8	5.2 KB
HT head	27.3	20.7 KB
Inter-row strips	4.0	44 KB

The stall is in A1's first half — the fused QKV column all-reduce, not the queue-7 band broadcast

A split probe on the compile-time-queue-7 tree, one device job, two minutes. Binding queue 7 at compile time changed 128,131 cycles per token to 128,139, so the raw-rebind theory is dead. Measured device probe; the six-step cycle is intact (four slow steps, one at 6.2K, one at 143.2K).

A1 HALF	WHAT IT DOES	SLOW STEPS	FAST STEP
First	RMSNorm, Q/K/V matvecs, QKV column all-reduce (256 rows)	151.5K cycles (median)	6.2K
Second	QK-norm band reduce (32 cols) + RoPE, on queue 7	1,856 cycles	1,856

A 128-square block is a quarter of today's block, so it is the L = 2 rung — the equal-area L = 4 is 256 wide by 128 tall

Le's catch: L = 4 keeps 4 of today's 9 layers per block (4/9 of the weights), but a 128-square block is 1/4 of a 256-square one. Those do not match. Fabric use, omitted below: today 4 rows of 256; both L = 4 variants 9 x 128 = 1,154 rows, the square one using only ~400 of 762 columns. Analytical correction.

QUANTITY	TODAY (A)	128-SQUARE, ONE LANE	256 X 128
PEs per block	65,536	16,384 (1/4)	32,768 (1/2)
Layers per block	9	4	4
Weight density per PE	1x	1.78x	0.89x
Active PEs total	524,288	294,912 (56%)	589,824 (1.13x)
Period if per-layer time is flat	107K cycles	~48K	~48K

At 32 rows a layer costs 1.48x the full-square time with one eighth of the PEs — 5.4x better than the linear prior

The first sub-height geometry ever to run on silicon: cut (i) at ATTN 32 rows | FFN 32 rows, one layer, 256-wide strips, 16,384 PEs total, 2K context. The identical geometry had wedged before the phantom-cell fix. Measured on-device TSC; rates at 0.85 GHz.

RUN	BUILD	CYCLES/TOKEN	TOK/S @0.85 GHZ
RB32_free	Free reference	—	1,000 tokens in 3 min
B1V_2K	Verify (full tail every step)	35,847	23,712 — tail-limited
B1P_2K	Production	17,589	48,326

Halving a block's rows from 256 to 128 costs 3.5%, to 32 costs 48% — while per-PE efficiency rises 5.4x

The height ladder, four points, one layer per block, 2K context, 256-wide strips, production builds, free references clean at every height. Nine minutes of wall time for four jobs. Measured on-device TSC.

ATTN FFN ROWS	PES	CYCLES/TOKEN	ATTN BUSY	TOK/S PER PE
256 256 (full square)	131,072	11,866	11,886	0.482
128 128	65,536	12,278	11,720	0.932
64 64	32,768	13,658	13,296	1.68
32 32	16,384	17,589	17,520	2.60

Every thin geometry is numerically sound: zero defects across 3,000 compared steps

The oracle gate for the height ladder. Byte-identity is impossible at sub-height because a strip's height changes the order in which column-wise reductions add partial sums, so the gate is: unordered top-k id set against an independent numpy implementation, with any differing member inside 3 bf16 ulp of the oracle's own k-th value, plus bounded value drift and determinism. Measured against a locally computed oracle.

HEIGHT	STEPS	ID SET IDENTICAL	WITHIN 3 ULP	DEFECTS
32 rows	1,000	138	862	0
64 rows	1,000	139	861	0
128 rows	1,000	139	861	0

L = 4 measured on the wafer: 2.08x today at 2K and 1.91x at 8K, with the flat-per-layer prior holding to 0.5%

The first full-model thin-stage deployment on silicon: 9 rows of ATTN 256x128 plus FFN 256x128, 4 layers per block, 18 stages, one lane, odd-row serpentine, funnel column. Production forced prefill. Measured on-device TSC; rates at 0.85 GHz.

CONTEXT	CYCLES/TOKEN	TOK/S @0.85 GHZ	VS A	PER-LAYER ATTN / FFN
2K	48,872	17,392	2.08x	13,018 / 6,024
8K	62,198	13,666	1.91x	17,665 / 6,027

L = 2 lands within 0.1% of the ladder prediction: 27,348 cycles per token, 3.72x today

One lane, 256x64 blocks, 2 layers per block, 18 rows, 36 stages — the same tree at H_ROWS = 64 with an even row count, so no odd-row return. Measured on-device TSC; rates at 0.85 GHz.

RUN	WSJOB	RESULT
Ladder prediction (2 x 13,658)	—	27,316 cycles/token
L2F_2K free reference	wsjob-anlkis2hlrjgrl3dkvumjg	1,000/1,000 steps; 852,657 cyc/token autoregressive
L2P_2K production forced	wsjob-y6j5dsywfuooutq7yrsequv	27,348 cyc/token = 31,081 tok/s

The whole ladder fits: even 72 stages leaves 15 KB per PE spare — the constraint is context, not SRAM

Settled by compiling it rather than arguing from arithmetic. Compiled per-PE maxima by fabric region, of a 48 KB bank, from CS-3 compile jobs. Inter-row strips, omitted below, are 4.3 KB/PE at every rung. Compiled (link-time), appliance toolchain.

REGION	L = 1	L = 2	L = 4
HT tail (binding, unchanged vanilla)	43.4	43.4	43.4
Block role A (ATTN)	33.1	32.0	31.2
Block role B (FFN)	28.2	27.9	27.8
HT head	27.3	27.3	27.3
Funnels	10.9 / 10.0	11.2 / 10.0	~10.5

Four rungs measured on silicon: the L axis does not create throughput, it converts context into stages

The deployable L ladder, one wafer, single request, forced prefill, 2K context, all four rungs measured on the device. Context caps are the pre-stride-lift figures (127 x block rows); after the lift they become compiled SRAM and stop halving. Measured on-device TSC; rates at 0.85 GHz.

LAYOUT	STAGES	CYCLES/TOKEN	TOK/S @0.85 GHZ	CONTEXT CAP (PRE-LIFT)
A: 4 rows x 256-square, 9 layers	8	101,670	8,360	29,184
L = 4: 9 rows x 256 x 128	18	48,872	17,392	16,256
L = 2: 18 rows x 256 x 64	36	27,348	31,081	8,128
L = 1: 36 rows x 256 x 32	72	17,771	47,831	4,064

Cut (iii) in place is worth 1.71x at nearly today's context ceiling; cut (ii) only 1.35x

The operation cuts placed inside today's allocation — one row is still two 256-square blocks carrying nine layers, cut into horizontal strips whose heights balance the stages. Predicted, from measured 256-square phase times and the measured height-scaling curve. Neither in-place cut was built.

LAYOUT	STRIPS (WEST EAST)	PACING STAGE	TOK/S @0.85 GHZ	CONTEXT CAP
Today, cut (i)	ATTN 256 FFN 256	ATTN 11,886	8,360 measured	29,184 measured
Cut (ii) in place	A1 64, A2 192 FFN 256	A2 8,375	11,273 predicted	24,384
Cut (iii) in place	A1 64, scan 192 O 48, F1 128, F2 80	scan 6,609	14,286 predicted	24,384

Only one of cut (iii)'s five stages touches the KV cache — which is why this axis keeps context

The five partitions and their computations. Every boundary is a contiguous run of measured phase slots from the phase-TSC profile, so no numerical restructuring is needed. Cycles per layer are measured on a full 256-square block at 2K, batch 1.

STAGE	WHAT IT OWNS	PHASES	CYCLES/LAYER
A1 — project	RMSNorm; Q/K/V matvecs; column all-reduce; QK-norm; RoPE	a + b	3,453
scan — attend (holds KV)	K/V write; score GEMV; softmax; P.V — all with their reduces	c + d + e	6,656
O-proj — finish attention	Output projection + X reduce; residual add and norm	f + g	1,778
F1 — expand	RMSNorm of Z; up/gate GEMVs + reduce; SiLU; SwiGLU	h + i	3,742
F2 — contract	Down GEMV + X reduce; residual; push to next layer	j	2,060

The FFN gives up half its rows for 4.3% and is never the pacing stage at any height

From the B-prime height ladder, which measured FFN strips at 256, 128, 64 and 32 rows alongside ATTN. The FFN is also completely context-independent (5,802 / 5,797 / 5,814 cycles at 2K / 8K / 23K), so none of this changes with sequence length. Measured on-device TSC, cycles per layer.

FFN ROWS	CYCLES/LAYER	VS FULL SQUARE	ATTN AT SAME HEIGHT
256	5,802	—	11,886
128	6,051	+4.3%	11,720
64	6,429	+10.8%	13,296
32	10,037	+73%	17,520

20.2 KB per PE sits idle in every FFN square — more capacity than the entire context we serve today

The FFN block's compiled SRAM is 27.8 KB of 48 per PE and does not grow with context (identical at 2K and 8K). Compiled SRAM (measured) plus arithmetic; the capacity is real but stranded without a streaming mechanism.

QUANTITY	VALUE
Idle FFN SRAM (262,144 PEs x 20.2 KB)	5.42 GB
KV for the whole model, per token	144 KB
Tokens of KV that idle SRAM could hold	~36,800
Today's measured on-chip context	29,184 (4.30 GB)

Compiled: at 128 attention rows the context ceiling really is the stride field, not SRAM

Appliance compile of the vertical-stack real config — one block column, six bands alternating ATTN 256x128 and FFN 256x64, three layers, 216,200 PEs. Compile job wsjob-fbzg6xly3yqk3lom4buuob, 12.6 s, cs1c 1.13.2. West and east strips, omitted below, are 4.2 KB/PE. Compiled per-PE maxima of a 48 KB bank.

REGION	KB/PE OF 48	PES
HT tail (unchanged, binding)	43.4	32,768
HT head (unchanged)	27.3	32,768
ATTN band, 256 x 128, one layer	25.5	32,768 each
FFN band, 256 x 64, one layer	19.4	16,384 each
Funnels (scatter / gather)	11.2 / 10.2	256 each

Standing a block in a column of its own costs its compute nothing — stacking's whole price is one strip hop

Vertical role stacking on silicon: one block column, six bands alternating ATTN 256x128 and FFN 256x64, 576 rows, one lane, three layers, 2K context. Production forced, block-TSC probe. Measured on-device TSC.

QUANTITY	MEASURED	HEIGHT-LADDER POINT	DELTA
Attention stage's own work, 256 x 128	11,568	11,720	-1.3%
Token period	12,202	pacing stage 11,720	+4.1%
FFN stage, 256 x 64	7,312	6,429	+13.7%
Rig throughput at 0.85 GHz	69,661 tok/s	—	rig, not a deployment

Side by side, measured: the L ladder trades context for throughput at a near-constant product (1 of 2)

Equal wafer area (589,824 PEs), one wafer, one request, 2K context, forced prefill. All four rows measured on the device. Context caps are the pre-stride-lift figures (127 x block rows). The stacked projections are on the next slide.

LAYOUT	STAGES	CYCLES/TOKEN	TOK/S @0.85 GHZ	CONTEXT CAP
Side by side, H = 256 (A)	8	101,670	8,360	29,184
Side by side, H = 128 (L = 4)	18	48,872	17,392	16,256
Side by side, H = 64 (L = 2)	36	27,348	31,081	8,128
Side by side, H = 32 (L = 1)	72	17,771	47,831	4,064

Stacked two lanes, projected: 1.34x at equal context, and nothing at equal heights (2 of 2)

The same equal-area comparison for stacked layouts. These rows are projections, not measurements: they apply the measured 4.1% hop premium to the measured height ladder. A one-lane stacked row is omitted deliberately — it uses half the columns (~73,203 cycles, ~11,612 tok/s) and is the trap the design note warned about.

LAYOUT	STAGES	CYCLES/TOKEN	TOK/S @0.85 GHZ	CONTEXT CAP
Stacked 2 lanes, ATTN 128 FFN 64	24	~36,606	~23,220	16,256
Stacked 2 lanes, ATTN 96 FFN 32	36	~26,042	~32,640	12,192
Stacked 2 lanes, ATTN 32 FFN 32	72	~18,238	~46,606	4,064

The stride cap is removed: 1,000 device records byte-identical, +0.05% cost, ceiling 4,064 -> 12,032

The one strided K-cache descriptor becomes four scalar stores — same values, same addresses, so byte-identity holds by construction. The ceiling is compiled SRAM and is pinned, not bracketed. A selection of 5 of 6 sweep points: 11,776 (368 positions, 47.3 KB) is omitted. Compiled sweep plus a device byte-identity gate.

MAX_SEQ_LEN	POSITIONS/PE	RESULT	ATTN ROLE A
2,048	64	fits (byte-identity point)	33.1 KB
4,096	128	fits — past the old 127 cap	36.1 KB
8,192	256	fits	42.1 KB
12,032	376	fits — the pinned ceiling	47.7 KB
12,064	377	ran out of PE memory for data	—

L = 1 stays above native prefill across its whole context range — 4.3x at 1.5K, still 1.4x at 11.5K

The throughput-versus-context curve after the stride lift: 36 rows of 256x32, 72 stages, production forced, each point's prefill sized to its context and 1,000 decode steps, so each point is a mean over a context window rather than a single depth. Fit: cycles/token = 12,225 + 3.626 x context, R-squared 0.99999. Measured on-device TSC.

MEAN CONTEXT	CYCLES/TOKEN	TOK/S @0.85 GHZ	VS NATIVE PREFILL
1,524	17,780.0	47,807	4.29-4.53x
3,572	25,177.6	33,760	3.03-3.20x
7,668	39,949.5	21,277	1.91-2.02x
11,508	54,000.3	15,741	1.41-1.49x

Every byte of the per-position cost is now named — and one of the five buffers is a free 17% of context

A grep for kv_len_per_pe in an array's declared size yields exactly five allocations; the earlier accounting counted three. The five total 48 B per position, against 47.9 measured. Only K and V scale with layers, at 16 B per position per layer; the other three total exactly 32 B and are layer-independent. Compiled and measured at L = 1.

BUFFER	B/POSITION/PE AT L = 1	SCALES WITH LAYERS?
XKCache_tile	8	yes
XVCache_tile	8	yes
score_f32	16	no
score (a bf16 copy)	8	no
kv_ingress_buf (staging)	8	no

The stride lift is worth 2.96x at L = 1 and nothing at A — A was always SRAM-bound, not stride-bound

All four rungs compiled and pinned: the listed ceiling compiles and the next admissible value on that rung's grid does not. The per-position slope test passes across a nine-fold range in layers per block. Compiled (link-time), compile-only jobs.

RUNG	OLD 18 CAP	COMPILED CEILING	LIFT	B/POSITION MEASURED
L = 1 (32 rows, 1 layer)	4,064	12,032	2.96x	47.9 (model 48)
L = 2 (64 rows, 2 layers)	8,128	18,176	2.24x	64.2 (model 64)
L = 4 (128 rows, 4 layers)	16,256	24,832	1.53x	96.0 (model 96)
A (256 rows, 9 layers)	32,512	26,112	none	180 (model 176)

The decoder passed the prefill kernel at L = 4 and is 4.3-4.5x past it at L = 1

Native prefill has not changed all session; everything on the other side of the comparison has. A selection of 5 of 7 configurations: rows-as-stages (4 stages, 177,539 cyc, 4,788 tok/s) and the projected stacked two-lane layout (~36,606 cyc) are omitted. Cycles measured; tok/s at 0.85 GHz.

CONFIGURATION	STAGES	CYCLES/TOKEN	TOK/S @0.85 GHZ	VS NATIVE PREFILL
Native prefill kernel	—	76,282 / 80,615	11,143 / 10,544	1x
Forced decode, ATTN FFN split	8	101,670	8,360	0.75-0.79x
L = 4, 256 x 128 blocks	18	48,872	17,392	1.56-1.65x
L = 2, 256 x 64 blocks	36	27,348	31,081	2.79-2.95x
L = 1, 256 x 32 blocks	72	17,771	47,831	4.29-4.54x